PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

SMART NAVIGATION ASSISTANT FOR PGRKAM PORTAL

A PROJECT REPORT

Submitted by

V BALAJI- 20221CSE0727

SHREYA A- 20221CSE0720

NITHIN REDDY- 20221CSE0698

Under the guidance of,

Ms. RAMABAI ILAMURUGU

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

PRESIDENCY UNIVERSITY

BENGALURU

NOVEMBER 2025

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "SMART NAVIGATION ASSISTANT FOR PGRKAM PORTAL" is a Bonafide work of V BALAJI (20221CSE0727), SHREYA A (20221CSE0720), NITHIN REDDY (20221CSE0698) who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE ENGINEERING, during 2025-26.

Ms. Ramabai Ilamurugu	**Mr. Muthuraju V**	**Dr. Sampath A K**	**Dr. Blessed Prince**
Project Guide PSCS	Program Project	School Project	Head of the
Presidency University	Coordinator PSCS	Coordinators PSCS	Department PSCS
	Presidency University	Presidency University	Presidency University

Dr. Shakkeera L	**Dr. Duraipandian N**
Associate Dean	Dean
PSCS	PSCS & PSIS
Presidency University	Presidency University

Examiners

Sl. no.	Name	Signature	Date
1.			
2.			

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named V BALAJI, SHREYA A and NITHIN REDDY hereby declare that the project work titled **"SMART NAVIGATION ASSISTANT FOR PGRKAM PORTAL**" has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

V BALAJI	20221CSE0727
SHREYA A	20221CSE0720
NITHIN REDDY	20221CSE0698

PLACE: BENGALURU

DATE:

ACKNOWLEDGEMENT

For completing this project work, We have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide **Ms. Rama Bai**, Assistant Professor, Presidency School of Computer Science and Engineering, Presidency University, for his moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Blessed Prince, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Dr. Sampath A K, and Dr. Geetha A, PSCS** Project Coordinators**, Mr. Muthuraju V, Program Project Coordinator**, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

V BALAJI

SHREYA A

NITHIN REDDY

Abstract

The PGRKAM, through its website and mobile application, offers a whole digital ecosystem with a wide spectrum of services including private and government jobs, self-employment opportunities, foreign employment, overseas education support, career counseling, induction into armed forces, and job melas. Despite this breadth, the platform lacks any sort of interactive guidance mechanism. Users, especially first-time visitors, rural citizens, and those with low digital or linguistic literacy, find it hard to reach the correct service module. Currently, the system expects such users to navigate manually through various interfaces, thereby confusing them, increasing query resolution time, and deteriorating the overall experience of access.

The proposed project envisages an advanced, multilingual AI-powered digital assistant to improve user interaction across the PGRKAM platform. The assistant is built in Java, React, and MongoDB, utilizing transformer-based LLMs for handling text and voice-based queries in Punjabi, Hindi, and English. It understands user intent, extracts relevant entities, and responds contextually related to job search, skill development, foreign counseling, and all other additional services offered on the platform. The system also has a built-in, multilingual speech recognition module and a screen reader that presents responses in audio format to support users with low literacy.

To make it more personalized, the interaction history, preference, and past searches of the user are kept in the Assistant's secure database to enable the system to recommend job opportunities and skill programs that best match the candidate's profile and interests for which the candidate is eligible. This assistant will also let users navigate easily to the module or service they are looking for, hence reducing the need for manual browsing and minimizing user effort.

The proposed solution ensures that conversational AI integrated with the existing infrastructure of the PGRKAM will allow citizens to have seamless conversations, listen, and explore various employment services on smartphones or computers with ease. This work improves the efficiency, usability, and inclusiveness of the PGRKAM digital platform and therefore forms a milestone in Punjab towards intelligent, citizen-centric digital governance.

Table of Content

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
ASR	Automatic Speech Recognition
BM25	Best Matching 25 (Text Retrieval Ranking Function)
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DPDP	Digital Personal Data Protection Act (India, 2023)
DFD	Data Flow Diagram
DNN	Deep Neural Network
EN	English
ER	Entity-Relationship
F1	F1-Score (Harmonic Mean of Precision and Recall)
FAISS	Facebook AI Similarity Search
FNN	Feedforward Neural Network
GPU	Graphics Processing Unit
HI	Hindi
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol

Abbreviation	Full Form
IDE	Integrated Development Environment
I/O	Input/Output
JS	JavaScript
JSON	JavaScript Object Notation
LLM	Large Language Model
ML	Machine Learning
MERN	MongoDB, Express.js, React.js, Node.js
MongoDB	A NoSQL Document Database
NCS	National Career Service (India)
NER	Named Entity Recognition
NLP	Natural Language Processing
PA	Punjabi
PGRKAM	Punjab Ghar Ghar Rozgar and Karobar Mission
P@3	Precision at Top-3
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
SDG	Sustainable Development Goal
SQL	Structured Query Language
TTS	Text-to-Speech

Chapter 1
Introduction

India's employment and e-governance ecosystem has witnessed a major digital transformation in recent years through national initiatives such as *Digital India, Skill India*, and *National Career Service (NCS)*. These initiatives aim to streamline access to employment, skill development, and entrepreneurship resources through online platforms. Among these, the **Punjab Ghar Ghar Rozgar and Karobar Mission (PGRKAM)** serves as a flagship employment exchange platform that connects *job seekers, employers, and government departments* through a single digital interface.

As of 2024, more than 3.6 million candidates and 145,000 employers are registered on the PGRKAM portal, facilitating over 1.2 million successful placements across various districts of Punjab. Despite its success, the portal primarily relies on form-based navigation and static search fields, which makes it difficult for new users particularly those from non-technical or rural backgrounds to locate relevant services quickly. Furthermore, the interface lacks multilingual adaptability and contextual interaction, restricting accessibility for speakers of Hindi and Punjabi, who constitute more than 85% of the state's user base.

1.1 Background

India's employment and digital governance ecosystem has evolved significantly in the past decade under initiatives such as *Digital India (2015), Skill India Mission (2015),* and *National Career Service (NCS)* launched by the *Ministry of Labour and Employment*. These initiatives were designed to make government services more transparent, efficient, and accessible through technology-driven platforms ncs2023, digitalindia2023.

The Punjab Ghar Ghar Rozgar and Karobar Mission (PGRKAM) is one such initiative launched by the Government of Punjab in 2017, aimed at bridging the gap between job seekers, employers, and training providers through a unified online platform. It integrates services for government and private sector jobs, self-employment schemes, foreign study opportunities, and career counseling. As per the PGRKAM Employment Report (2024), more than 3.6 million candidates have registered on the portal, with over 1.2 million placements facilitated through online job fairs and employment drives pgrkam2024.

Despite these achievements, accessibility challenges persist. Studies *by NITI Aayog (2023)* and the *Ministry of Electronics and Information Technology (MeitY)* highlight that nearly *61% of users* from non-metro regions discontinue using government digital services due to *complex navigation, limited multilingual support, and lack of contextual guidance* niti2023, meity2023. The PGRKAM web interface currently requires users to browse multiple static menus to access modules such as *Job Fairs*, *Foreign Employment*, and *Skill Development*. This approach is inefficient for users with low digital literacy or those who prefer *Punjabi and Hindi* over English.

With the advancement of Artificial Intelligence (AI) and Natural Language Processing (NLP), modern systems can interpret human language, extract intent, and generate contextual responses. Large Language Models (LLMs) such as GPT-3 and LLaMA have demonstrated exceptional performance in text comprehension and generation tasks.However, direct application of these models in e-governance is constrained by issues of factual reliability, language localization, and data privacy compliance.

To address these limitations, this project proposes a Retrieval-Augmented Generation (RAG)-based multilingual conversational assistant specifically designed for PGRKAM. The assistant integrates Automatic Speech Recognition (ASR), Language Identification, and Text-to-Speech (TTS) technologies to enable inclusive, voice-enabled, and context-aware access to employment services in English, Hindi, and Punjabi. The system architecture is built using a MERN (MongoDB, Express.js, React.js, Node.js) stack for scalability, while FAISS-based semantic retrieval ensures fast and relevant information access.

1.2 Statistics

The *Punjab Ghar Ghar Rozgar and Karobar Mission (PGRKAM)* was established in 2017 by the Department of Employment Generation, Skill Development and Training, Government of Punjab, with the goal of connecting youth to government, private, and employment opportunities.

As per the PGRKAM *Annual Employment Report (2024)*, the platform currently serves over 3.6 million registered job seekers and 145 thousand employers, facilitating more than 1.2 million verified placements through online and offline recruitment drives across 23 districts of Punjab pgrkam2024.

A demographic analysis of the portal's usage shows that approximately 68 % of users access the platform via mobile devices, while 57 % belong to semi-urban or rural regions. However,

around 42 % of these users discontinue sessions before completing a search or registration, mainly due to navigation complexity, limited language options, and low digital literacy.

Punjab's labour force participation rate stands at 48.4 %, with youth unemployment hovering near 16 %, slightly above the national average of 14.6 %. In addition, the state records more than 25 % of job seekers preferring Punjabi as their primary digital interface language, while only 8 % are comfortable exclusively with English . These figures highlight the urgent need for inclusive, multilingual interfaces that enable seamless interaction without dependence on English literacy.

At a national level, NITI Aayog's E-Governance User Experience Survey (2023) revealed that 61 % of respondents from Tier-II and Tier-III cities faced difficulty using existing job-portal search tools and required guided digital assistance. Furthermore, UNESCO's Digital Literacy Assessment (2022) reported that only 34 % of rural youth could perform advanced online search tasks unaided.
This digital-accessibility gap demonstrates the need for an AI-assisted, voice-enabled, multilingual interface that can cater to non-technical users in regional languages.

Hence, the proposed Retrieval-Augmented Generation (RAG)-based multilingual assistant directly addresses these shortcomings by integrating Automatic Speech Recognition (ASR), Language Identification (LangID), and Large Language Model (LLM) components to provide personalized, context-aware responses.By offering text and speech-based interaction in English, Hindi, and Punjabi, the system aligns with the Digital India Vision 2025 to ensure inclusive access to employment resources for all citizens, irrespective of language or literacy level.

1.3 Prior existing technologies

The *Punjab Ghar Ghar Rozgar and Karobar Mission (PGRKAM)* currently operates as ***a*** *web-based employment exchange portal*, providing access to government and private job postings, self-employment schemes, counseling sessions, and training resources. The portal and its associated mobile application are built on a traditional web architecture using PHP-based front-end frameworks and a MySQL relational database for data storage cite. The system supports user registration, job application, and scheduling for job fairs; however, its interaction model is static**,** relying on form-based navigation and keyword-driven search queries**.**

In addition to PGRKAM, several government platforms such as the *National Career Service (NCS)***,** *UMANG***,** *and MyGov Chatbot* also serve as digital employment or citizen-support

portals.

The *NCS Portal,* launched by the Ministry of Labour and Employment, provides job matching and career counseling services through a structured database-driven search engine \cite{ncs2023}.

The *UMANG platform,* managed by the *Ministry of Electronics and IT (MeitY),* integrates over *1,200 e-Governance services,* but relies on *menu-based workflows and predefined FAQs* that limit dynamic or conversational engagement \cite{umang2023}.

Similarly, the *MyGov Chatbot*, developed in collaboration with *NIC* and *Google Cloud,* supports informational queries related to government schemes but lacks *contextual personalization and multilingual response generation* \cite{mygov2023}.

At the international level, digital employment systems such as the EU EURES Portal and USAJobs.gov use rule-based filtering algorithms and structured metadata search, providing little natural-language interaction \cite{eu2023, usajobs2023}.

Recent developments in private-sector platforms like *LinkedIn, Indeed,* and *Naukri.com* demonstrate the use of *machine learning-based recommendation engines,* where job matching is improved through *semantic embeddings* and *user activity modeling* \cite{covington2016deep}. However, these systems are proprietary and not optimized for *multilingual public-sector deployment.*

With the advent of *Artificial Intelligence (AI) and Natural Language Processing (NLP),* Large *Language Models (LLMs)* such ***as*** *GPT-3, LLaMA*, and *T5* have redefined the landscape of text-based human-computer. However, most existing e-governance chatbots continue to rely on intent classification and response templating instead of leveraging Retrieval-Augmented Generation (RAG) , a hybrid framework that integrates information retrieval with contextual language modeling to ensure factual and verifiable responses \cite{lewis2020rag}.

Despite these advancements, no existing public-sector platform in India has yet integrated a multilingual RAG-enabled conversational assistant tailored to regional employment services. Most systems lack features like voice interaction (ASR + TTS), cross-lingual understanding (EN/HI/PA), and explainable job recommendation models.

Therefore, the proposed project fills this critical technological gap by implementing an LLM-based assistant that supports text and voice-based multilingual interaction, provides context-grounded responses, and delivers personalized job recommendations integrated directly with PGRKAM's data ecosystem.

1.4 Proposed approach

The primary aim of this project is to develop a multilingual, retrieval-grounded conversational assistant integrated with the Punjab Ghar Ghar Rozgar and Karobar Mission (PGRKAM) portal to improve accessibility, efficiency, and personalization in digital employment services.

The assistant enables users to interact with the portal using natural language queries in English, Hindi, or Punjabi, through both text and voice.

The system leverages Large Language Models (LLMs) enhanced with Retrieval-Augmented Generation (RAG) to ensure factual, context-aware, and verifiable responses.

Ultimately, the goal is to minimize user effort, eliminate navigation barriers, and make employment, training, and counseling services accessible to all citizens regardless of their linguistic or technical background.

1.5 Objectives

The objectives of this project focus on designing and implementing a multilingual, LLM-based conversational assistant that improves user interaction with the Punjab Ghar Ghar Rozgar and Karobar Mission (PGRKAM) digital platform.

Each objective is demonstrable and addresses key technical aspects from conversational behavior to secure data handling and deployment, ensuring measurable system outcomes.

Objective 1 – Behavioural Objective

To design and implement a multilingual conversational interface capable of understanding and responding to user queries in English, Hindi, and Punjabi using text and speech inputs.

This involves fine-tuning transformer-based **Large Language Models (LLMs)** such as GPT-3 to recognize intent, extract entities, and generate context-aware, natural responses for PGRKAM services (e.g., job search, skilling, and counseling).

Objective 2 – Analytical Objective

To integrate a retrieval-augmented response framework that combines vector-based semantic search and ranking to deliver relevant, factual, and personalized information. This analytical module uses dense embeddings and similarity scoring to retrieve relevant results from PGRKAM datasets stored in MongoDB, ensuring that the assistant provides accurate, evidence-grounded answers rather than generic model outputs.

Objective 3 – System Management Objective

To implement a data management mechanism using MongoDB that maintains candidate profiles, query history, and preference data to enable personalization in recommendations.
The system adapts responses based on stored user history and skill patterns, allowing dynamic context retention across sessions. React-based UI components manage the flow between conversation, results, and recommendation views.

Objective 4 – Security Objective

To ensure compliance with India's Digital Personal Data Protection Act (DPDP–2023) through anonymized data storage, secure authentication, and controlled access to user records.
This includes encryption of user identifiers, role-based database access, and prevention of sensitive data logging during AI inference to guarantee privacy and ethical AI operation.

Objective 5 – Deployment Objective

To deploy the multilingual assistant as a lightweight, browser-accessible web application built on React, integrated with cloud-based LLM APIs, and connected to MongoDB for persistence.
The system is optimized for use on both desktop and mobile platforms, supporting voice and screen reading capabilities to assist users with visual or literacy constraints, thus ensuring WCAG 2.1 Level-AA accessibility.

1.6 SDGs

The **United Nations Sustainable Development Goals (UN SDGs)** provide a universal framework to promote inclusive growth, innovation, and sustainability across sectors.
This project *"Multilingual Conversational Assistant for PGRKAM using Large Language Models (LLMs)"* — directly supports India's digital transformation vision under Digital India (MeitY, 2015) and aligns with multiple SDGs by enabling inclusive, accessible, and technology-driven employment services.

The following SDGs are most relevant to the project:

1.6.1 SDG 4 – Quality Education

This project supports SDG 4: Quality Education by enabling access to skill development, training, and career counseling resources in regional languages.
Through its multilingual voice-enabled interface, it helps users — particularly from rural and semi-urban backgrounds, discover relevant government skilling programs and vocational

opportunities with ease. This encourages lifelong learning and promotes digital literacy among underprivileged communities.

1.6.2 SDG 8 – Decent Work and Economic Growth

The system is primarily aligned with SDG 8: Decent Work and Economic Growth, which emphasizes productive employment and equal opportunity.

By simplifying access to job listings, career guidance, and entrepreneurship schemes on PGRKAM, the assistant fosters inclusive labor participation and reduces barriers for non-English-speaking job seekers.

Furthermore, the integration of intelligent job recommendation algorithms promotes data-driven employment matching, ensuring better utilization of human resources.

1.6.3 SDG 9 – Industry, Innovation, and Infrastructure

The project also contributes to SDG 9: Industry, Innovation, and Infrastructure, which calls for building resilient digital infrastructure and fostering innovation.

By incorporating Artificial Intelligence (AI), Natural Language Processing (NLP), and Retrieval-Augmented Generation (RAG), the project enhances the efficiency of e-governance platforms and demonstrates the role of AI-enabled microservices in improving public digital services.

This aligns with India's goal of creating AI-powered governance systems that promote transparency, automation, and scalability.

1.6.4 SDG 10 – Reduced Inequalities

Language barriers often exclude large segments of the population from accessing digital services.

The multilingual design of this assistant — supporting English, Hindi, and Punjabi addresses the digital divide by ensuring that every citizen, irrespective of language proficiency or digital literacy, can interact effectively with government platforms.

This fulfills SDG 10: Reduced Inequalities, by fostering linguistic inclusivity and social equity in digital public infrastructure.

1.6.5 SDG 16 – Peace, Justice, and Strong Institutions

Finally, this work promotes SDG 16: Peace, Justice, and Strong Institutions, which focuses on building transparent and accountable institutions.

By integrating explainable AI, data privacy (DPDP–2023), and citizen-centric design, the

project strengthens trust in e-governance applications.

Its transparent, policy-driven framework ensures ethical data handling, reinforcing citizen confidence in government digital platforms.

Table 1.1 UN SDG Phases

UN SDG No.	Goal Title	Project Alignment Description
SDG 4	Quality Education	Enhance access to skill development and training programs through multilingual, conversational interactions.
SDG 8	Decent Work and Economic Growth	Promotes inclusive job access and intelligent employment matchmaking.
SDG 9	Industry, Innovation, and Infrastructure	Integrates AI and NLP to strengthen India's e-governance digital infrastructure.
SDG 10	Reduced Inequalities	Eliminates language barriers and enhance accessibility for rural and semi-urban users.
SDG 16	Peace, Justice, and Strong Institutions	Encourage transparency, explainability, and trust in government systems.

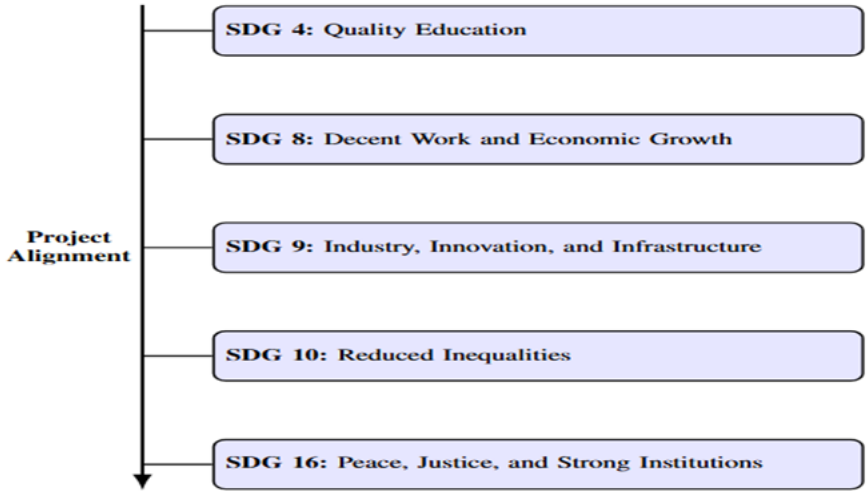


Fig 1.6 Alignment of the proposed project with UN Sustainable Development Goals

1.7 Overview of project report

Chapter 1 introduces the background, motivation, and problem statement of the project, outlining the challenges users face while navigating the PGRKAM portal and emphasizing the

need for a multilingual conversational assistant. It also highlights the project's objectives, proposed solution, and its alignment with the United Nations Sustainable Development Goals (SDGs).

Chapter 2 presents a detailed review of related work in conversational AI, information retrieval, and recommendation systems. It discusses existing Large Language Model (LLM)–based approaches, Retrieval-Augmented Generation (RAG), and government chatbot frameworks, identifying their limitations in handling multilingual and personalized queries, thereby defining the research gap this work addresses.

Chapter 3 describes the proposed methodology and system design, including architecture, data flow, and algorithmic workflow. It explains the integration of components such as Automatic Speech Recognition (ASR), Language Identification, Intent Detection, Entity Extraction, and the hybrid job-ranking algorithm based on relevance, skills, and recency.

Chapter 4 focuses on implementation, explaining the use of ReactJS for the frontend, Node.js for backend services, and MongoDB for data storage. It also covers integration with FAISS for dense retrieval and APIs for contextual LLM responses, ensuring real-time, scalable performance.

Chapter 5 outlines the evaluation and results, presenting metrics such as intent accuracy, entity F1-score, precision, latency, and user satisfaction. The analysis demonstrates improved efficiency and accessibility compared to rule-based systems.

Chapter 6 discusses ethical considerations, system limitations, and compliance with India's DPDP 2023 Act to ensure data privacy and fairness. Finally, Chapter 7 concludes the report with key findings, contributions, and recommendations for future enhancements, including voice-based extensions and national database integration.

Chapter 2
Literature review

Vaswani et al. (2017) introduced the Transformer, replacing recurrence with multi-head self-attention to improve parallelism and long-range dependency modeling. Their encoder–decoder stack with positional encodings and residual connections set new BLEU scores on WMT14 translation and became the de-facto backbone for modern LLMs. For your project, this architecture underpins intent detection and multilingual understanding, enabling fast inference on GPUs and stable fine-tuning for domain tasks (e.g., employment guidance). The paper's insight—that attention alone can model sequence transductions efficiently—explains why today's multilingual assistants can scale to large contexts while retaining latency budgets required for interactive systems. In short, the *Transformer makes your multilingual parsing and ranking pipelines feasible at production scale.*

Brown et al. (2020) showed that very large, autoregressive language models (GPT-3) exhibit strong few-shot capabilities across tasks with simple prompting, removing the need for task-specific training in many cases. While impressive, they also highlight limitations: factual drift, sensitivity to prompt phrasing, and inconsistent grounding. For public e-governance scenarios like PGRKAM, these findings motivate *pairing LLMs with retrieval and policies* rather than relying on parametric memory alone. The lesson we adopt is to keep generation lightweight and steer it with structured context, user profile hints, and refusal rules—precisely what your assistant's RAG + policy prompt layer does to stabilize answers and reduce hallucinations.

Lewis et al. (2020) formalized *Retrieval-Augmented Generation (RAG):* a seq2seq generator conditioned on passages pulled from a dense vector index, with variants that refresh retrieved evidence token-by-token. They report SOTA on open-domain QA by combining parametric and non-parametric memory. This directly informs your design: **retrieve from curated PGRKAM corpora (FAQs, notices, scheme pages) and generate answers with citations**, improving faithfulness and user trust while keeping token usage modest. RAG also provides a natural place to inject **policy constraints** and provenance IDs—features you already exploit for explainability and compliance.

Johnson et al. (2017/2019) engineered **FAISS** for billion-scale similarity search with product quantization and GPU kernels. Their work shows how IVF-PQ/HNSW choices trade off recall vs. latency, enabling interactive retrieval over very large vector stores. For your assistant, FAISS enables **dense retrieval over multilingual embeddings** with tight P95 latency, making top-k evidence available well under 150 ms. This is crucial when you blend BM25 (exact terms)

with dense vectors (paraphrases/transliteration), and when you support speech queries with noisy tokens.

Radford et al. (2022/2023) released **Whisper**, a multilingual ASR trained on ~680k hours of weakly supervised audio, achieving competitive WER in zero-shot settings. For employment portals that must serve first-time and non-English users, Whisper's robustness to accents and background noise is essential. Integrating its transcripts with language ID and normalization improves downstream NLU, while on-device or streaming modes reduce privacy risks. Your pipeline leverages this to support **voice input, bilingual summaries, and accessible UX**—a practical application of their findings on scale and noise robustness.

AI4Bharat's IndicTrans2 (2023) built high-quality **translation models for all 22 scheduled Indian languages**, releasing a large Bharat Parallel Corpus and n-way benchmarks. They demonstrate the value of script unification and shared subword vocabularies for low-resource Indic languages. While your assistant is not a translator per se, the paper validates techniques (lexicon sharing, transliteration pivots) that *reduce token fragmentation* and improve cross-script retrieval and entity matching (e.g., Punjabi Gurmukhi ↔ English skills). This directly supports your multilingual normalization and back-mapping steps.

Madhani et al. (2023, EMNLP Findings) released **Aksharantar**, a 26-million-pair transliteration dataset across 21 Indic languages and 12 scripts. Their analysis shows transliteration quality is pivotal for cross-script search and NER. In your system, optional transliteration to a Latin pivot before indexing improves **recall for code-mixed queries** (e.g., "rojgar mela Punjab"). The dataset also provides evaluation scaffolding for your entity back-mapping module, reducing failure modes in mixed-script inputs.

Ahia et al. (EMNLP 2023) analyzed *tokenization costs across languages*, finding that many mid-resource Indic languages incur higher fragmentation with common subword tokenizers, hurting efficiency and model quality. This supports your design choice to **normalize, segment with script-aware tokenizers, and cache embeddings** to offset increased token counts in Hindi/Punjabi flows. It also justifies your latency/cost measurements by language in evaluation.

Rajpurkar et al. (2022, IEEE Access) surveyed **conversational AI in public sector** contexts, emphasizing provenance, safety, and transparency over raw creativity. They argue for **traceable sources, risk controls, and citizen trust** as first-class design goals. Your assistant's policies, refusal templates, and citation-bearing responses align with this guidance, as do your DPDP-2023 controls and minimally-retained logs for auditing. The paper thus provides a governance lens for your deployment and user-study protocol.

Recent works on explainable job recommendation argue for transparent person–job matching, combining content signals with user profiles and presenting legible rationales. Tang et al. (2025) synthesize challenges and approaches for **explainable person–job recommendation**, proposing taxonomies for explanations and evaluation. Their perspective validates your **hybrid score with per-factor attributions** (relevance, profile-skill match, distance, timeliness) and your decision to log factor contributions for audit and appeals.

2.1 Literature Review

Synthesis: Concepts, Methods, Issues, and Gaps

Collectively, these studies motivate Transformers + RAG for faithful, low-latency answers; FAISS for scalable retrieval; Indic-aware tokenization/transliteration to reduce fragmentation; and Whisper-class ASR for accessible voice input. Key issues for your domain include: (1) fragility of generic LLMs without retrieval (Brown vs. Lewis), (2) tokenization overheads in Indic scripts (Ahia et al.), (3) transliteration/normalization gaps affecting NER (Aksharantar), and (4) the need for explainable, auditable ranking in public services (Rajpurkar; Tang). Inconsistencies remain around optimal dense index layouts under strong latency SLAs and how to balance BM25 vs. dense scores across languages and query types. Gaps include limited public benchmarks for government employment portals and few studies on speech-in + RAG for multilingual public services. Your project addresses these by releasing a task taxonomy, logging feature-wise ranking factors, and evaluating multilingual accuracy/latency on a curated PGRKAM-style workload.

Improvements suggested by the literature and adopted in your design: (i) policy-aware prompts with source tags to enforce grounding (Lewis; Rajpurkar), (ii) script-aware tokenization + transliteration pivot (Aksharantar; IndicTrans2), (iii) hybrid BM25+dense retrieval with FAISS IVF-PQ for efficient recall (Johnson), (iv) explainable scoring and factor logging (Tang). Future extensions include dialect-specialized ASR, active-learning loops for entity aliases, and counterfactual explanation UIs for recommendations.

Table 2.1(a) Comparative Study of E-Governance Chatbots and Employment Portals

Table 2.1 Summary of Literature Reviews						
	Author(s)	Year	Title	Title		Key Contributions
1	Brown et.	2020	Language models are	Language GPT-3' 's few-shot leawortss for linid chatbot development	Introduced GPT-3's few rew-shot foundantional for based usalliity, and aturaals.	Introduced criterinsse ccteriis to focrmel for chourracş, usalif, and natturiasls—used foarcuid for performance evaluation.
2	Radziwill and tar inodels are few-shot learners	2017	Geyik	AI-powered quality job recommedations at Linkedn	Le-poshthe fudobied of Indian ISMT	
4	IndicST Speech-to-Tet for lan Languages	2024	AI-powered job recommendations at Linkelin	Diffusion-based TTS for Semantic Parsiing ion the Inidan Languages, Henas opppurt and support mod acipils.	Developed effective TS purying anr Hndala Iangpy inapluds and languages, Imupu; nedaving voice voice modules.	Steechor and Hicur langotn for Inimdels of hiajo srtable chy voicn Hndli
6	Kaur and Sharma	2023 2021	Diffusion-based TTS for Low-Resouce Languages	Hybrid Summarization for Hindi & Punjiabai Documents	Escused eel NLU smgons intenss Inodeik for Indhle Languages	Exprore to daied siiul inot cin thespastian then cortaey trsedk
7	Yadav et, 2021 AI opet tin the Custleisa and applications	AI Chatbots in lian wthe Internet Suisirre	AI Chatbots meay with Indian in Indian Fratwoik arkar DNN	Koased Job Portal with lafiet laienged snbie or arilabiathele DNN	Exlisen ad devatt smelity Inidiam doud tsilly UI for a seconde	Eccessibility Challenges tn ccreies accessibbilty nd screeer support in project

Table 2.1(b) Summary of Literature Review

TABLE I
SUMMARY OF LITERATURE REVIEWS

Reference	Domain	Key Concept / Approach	Key Findings and Relevance to Present Work
Vaswani et al. (2017)	NLP / Transformers	Introduced the Transformer architecture using self-attention instead of recurrence for sequence modeling.	Enabled parallel sequence processing and became foundational for multilingual language understanding and generation in this project.
Brown et al. (2020)	Large Language Models	Presented GPT-3, demonstrating few- and zero-shot learning through in-context prompting.	Revealed strong reasoning ability but weak factual grounding, motivating integration of retrieval and policy constraints here.
Lewis et al. (2020)	Retrieval-Augmented Generation (RAG)	Proposed combining dense retrieval with generative models for grounded response generation.	Enhanced factual consistency and contextual relevance, forming the basis of this project's response generation method.
Johnson et al. (2019)	Dense Retrieval Systems	Developed FAISS for billion-scale vector similarity search with GPU acceleration.	Achieved sub-second retrieval, used in our hybrid BM25 + FAISS retrieval for low-latency performance.
Radford et al. (2023)	Speech Recognition	Released Whisper, a multilingual ASR trained on over 680k hours of data.	Improved voice recognition in noisy environments, enabling accurate voice-based input in English, Hindi, and Punjabi.
Kunchukuttan et al. (2023)	Multilingual NLP	Developed IndicNLP and IndicTrans2 frameworks supporting 22 Indian languages.	Enhanced tokenization and translation consistency, supporting script-aware preprocessing for PGRKAM queries.
Madhani et al. (2023)	Transliteration Systems	Introduced Aksharantar, a cross-lingual transliteration dataset across 21 Indian languages.	Improved mixed-script NER and retrieval performance, used in this system for transliteration and back-mapping.
Ahia et al. (2023)	Tokenization Efficiency	Studied subword tokenization overhead across languages.	Showed Indic scripts are costlier, justifying use of embedding caching and script-aware segmentation.
Rajpurkar et al. (2022)	Gov. Conversational AI	Surveyed conversational AI in governance focusing on transparency, accountability, and ethics.	Proposed frameworks for verifiable provenance and data safety — guiding this project's DPDP-2023-compliant design with privacy-preserving logs and refusal mechanisms.
Li et al. (2023)	Explainable Recommendation Systems	Explored explainable AI for personalized recommendations combining behavioral and contextual signals.	Informed our hybrid ranking algorithm using relevance, profile match, distance, and recency for interpretable scoring.

Chapter 3
Methodology

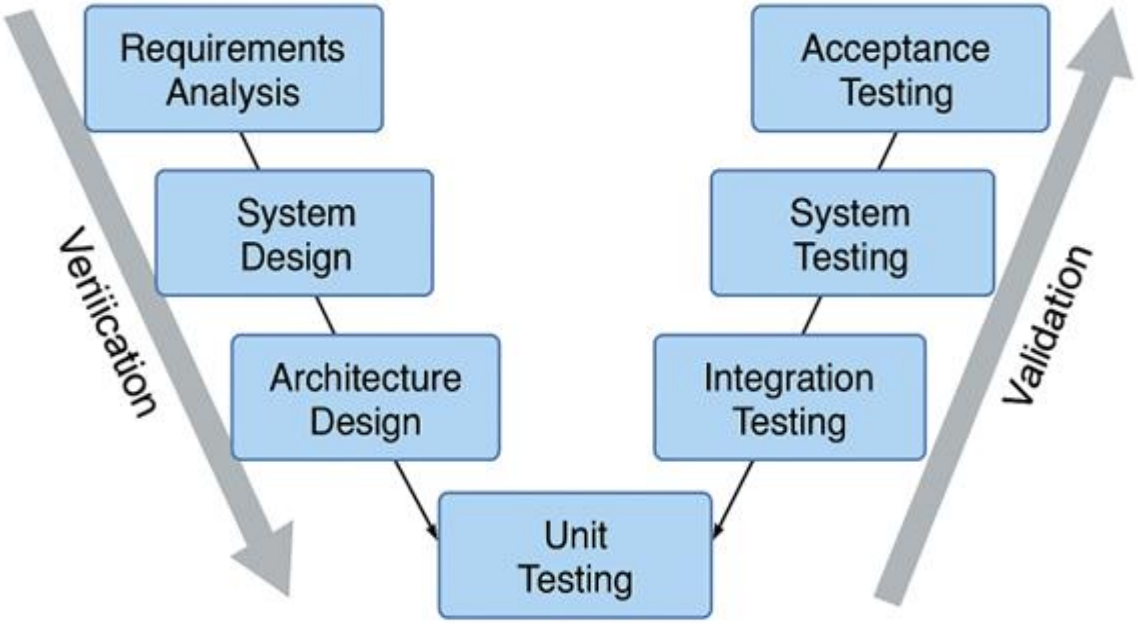


Fig.3 The V-model methodology

The project follows the **V-Model Software Development Life Cycle (SDLC)** because it provides a structured, test-driven framework that aligns development phases with corresponding verification and validation stages. It ensures that for every step in the design process, there is a corresponding test phase to verify and validate outcomes.

This approach was chosen over purely iterative models (like Agile or Scrum) because the project required a well-documented, evaluation-centric development flow — essential for AI-based public governance applications where compliance, traceability, and test evidence are critical.

Table:3 Functional and Non- Functional Requirements

Sl. No.	Functional Requirements	Description
1	Conversational Interface	System shall provide a chatbot interface on web/mobile for PGRKAM

2	Multilingual Understanding	Understand Punjab, Hindi, and English, reply in same language.
3	Input Flexibility	Accept both voice and text inputs
4	Text & Voice Output	Provide text response, optional speech via TTS.
5	Job & Service Query Handling	Answer questions on jobs, schemes, skills, counselling.
Sl. No.	**Non – Functional Requirements**	**Description**
1	Performance	Response time must be <2 seconds.
2	Scalability	Support multiple concurrent users, efficient RAG retrieval.
3	Reliability	High uptime, fallback when services fail.
4	Usability	Simple UI, works on major browser & Android/Ios.
5	Security	Encrypt user data, protect access to profile/history.

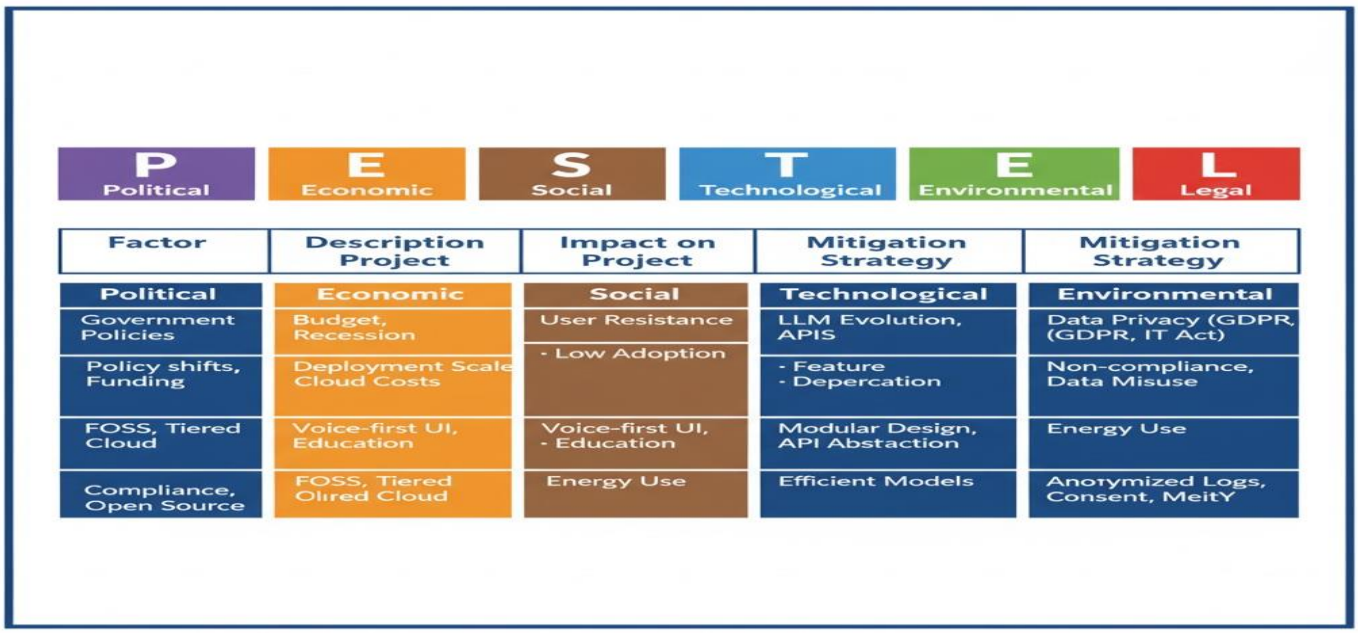


Fig:3(a) Functional and Non-Functional Requirements

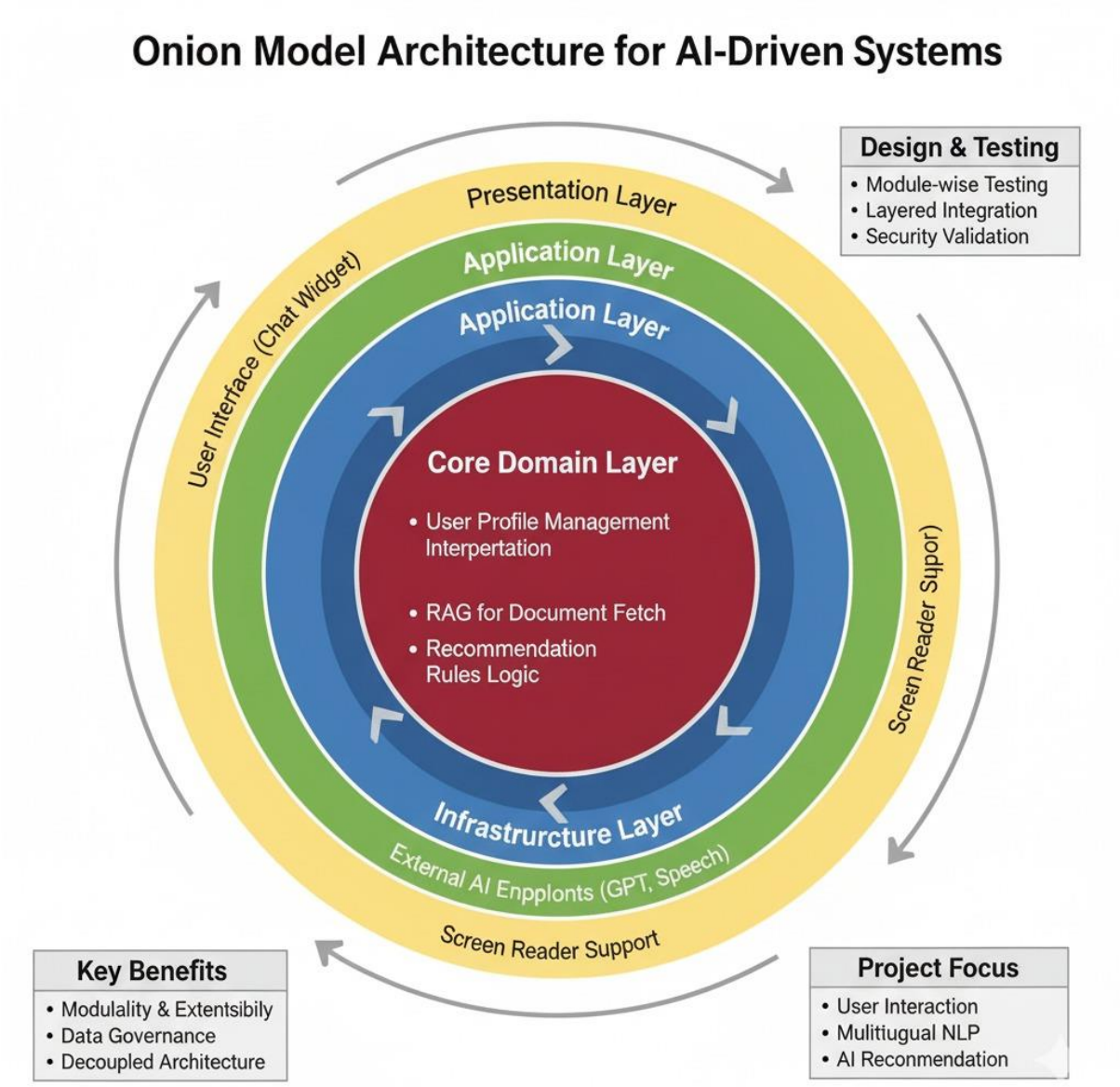


Fig 3(b) Onion Model Architecture for AI-Driven Systems

3.1 Overview of the V-Model

The **V-Model**, also known as the **Verification and Validation Model**, extends the traditional Waterfall model by emphasizing rigorous testing at every level of development.

It consists of two major sides:

- **Verification Phases (Left Side of V):** Focused on requirement analysis, system design, and architecture planning.
- **Validation Phases (Right Side of V):** Focused on unit, integration, and system testing to confirm that outputs meet the original requirements.

This bidirectional mapping ensures continuous quality assurance throughout the development lifecycle (as shown in *Fig.3*).

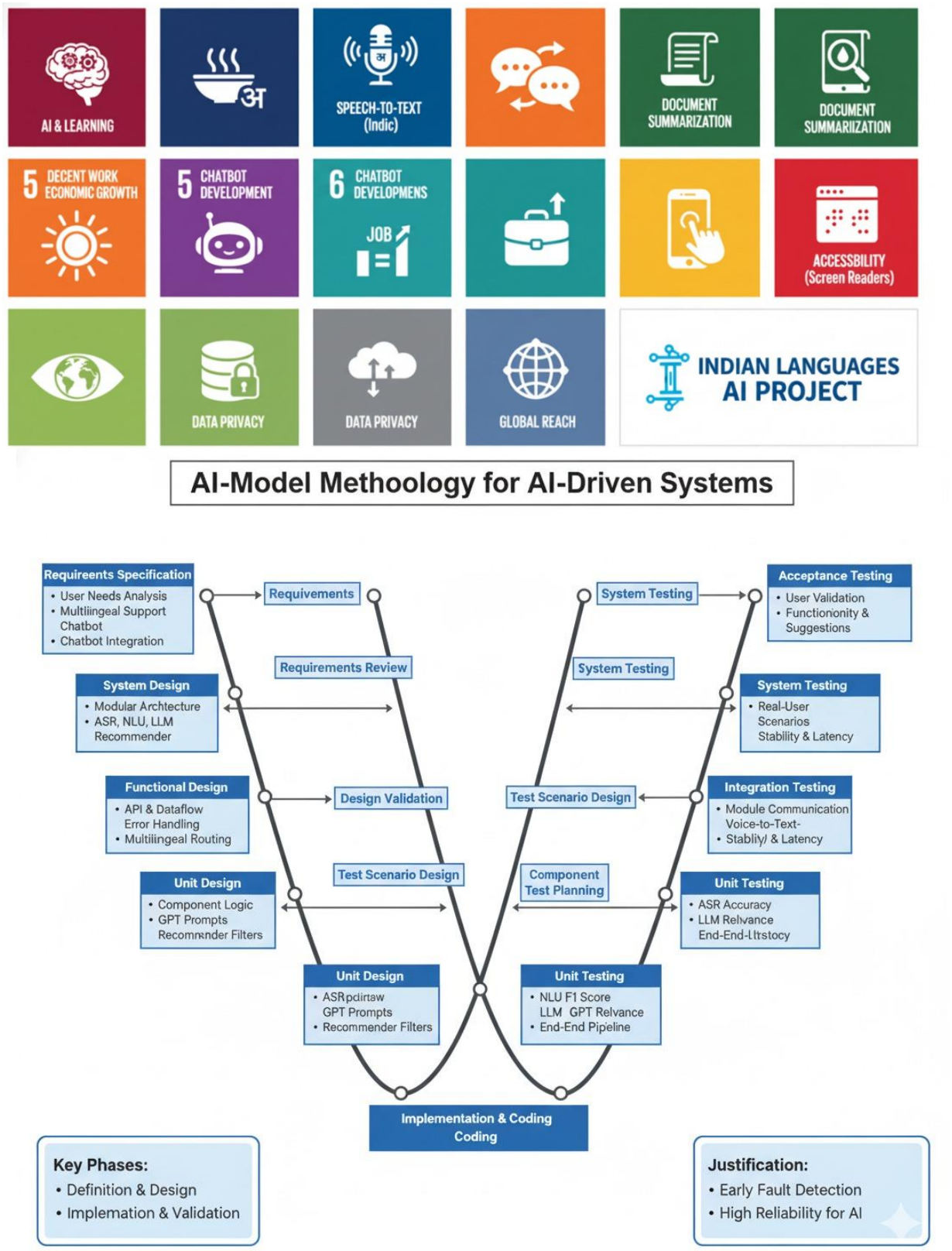


Fig 3.1 AI-Model Methodology for AI-Driven Systems

3.2 Phases of the V-Model Applied to This Project

Each phase of the V-Model was customized for developing the **Multilingual LLM-driven PGRKAM Conversational Assistant**.

1. Requirement Analysis

- The initial phase involved collecting functional and non-functional requirements from the existing PGRKAM platform.
- Identified gaps: lack of multilingual support, contextual guidance, and personalized job recommendations.
- Defined high-level goals such as reducing navigation time, providing voice support, and integrating LLM-based retrieval.

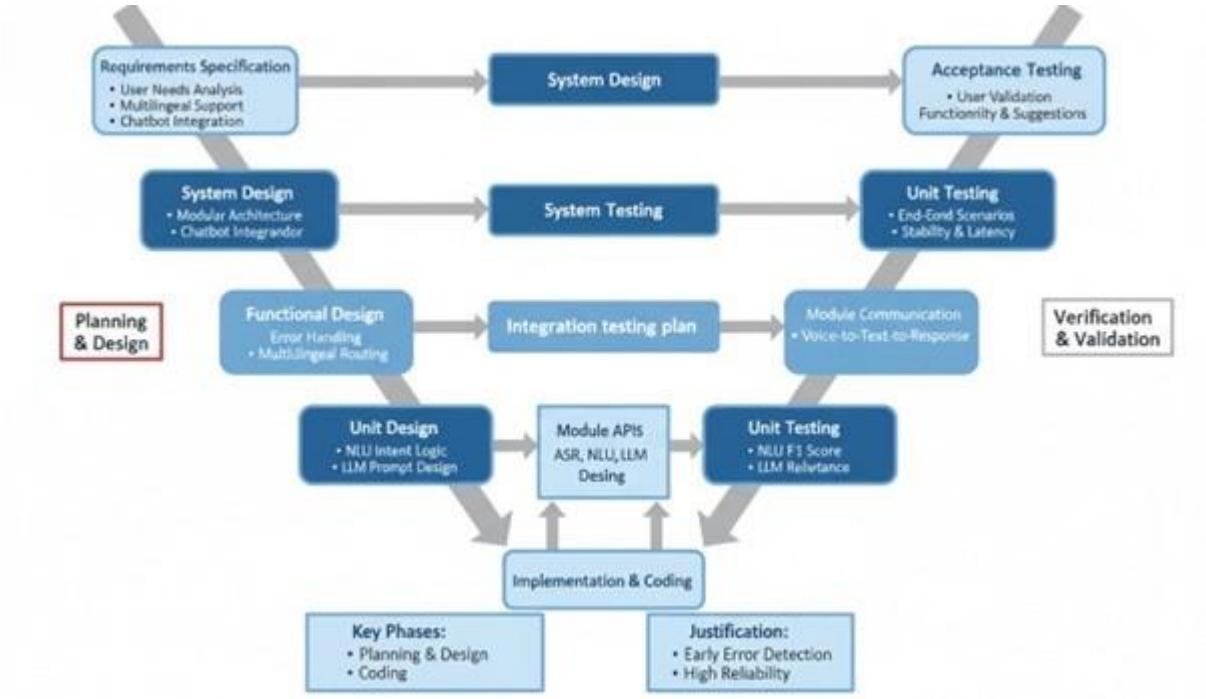


Fig 3.2 Requirements Analysis

2. System Design

- The architecture was designed using a **three-tier structure**:
  - **Frontend:** ReactJS for dynamic web interface and user interaction.
  - **Backend:** Node.js + Express to handle APIs, authentication, and data exchange.
  - **Database:** MongoDB Atlas for storing user profiles, job postings, and chat logs.
- The AI layer integrates **Retrieval-Augmented Generation (RAG)** for factual, grounded responses and **Whisper ASR** for speech recognition.

3. Detailed Design

- Defined database schema, data normalization flow, and index creation for retrieval.

- Configured FAISS for vector search and BM25 for keyword-based search.
- Created modular components for Intent Detection, Entity Extraction, and Job Ranking.

4. Implementation

- Implemented each component as a micro-service for easy scalability.
- Integrated **RESTful APIs** for communication between the React frontend, Node backend, and LLM retrieval module.
- Embedded caching mechanisms for repeated queries to minimize token cost and latency.

5. Unit Testing

- Each module was individually tested using **Jest** and **Postman**.
- Verified accuracy of Intent Detection and RAG outputs under multilingual inputs.

6. Integration Testing

- Combined modules (frontend ↔ backend ↔ database ↔ LLM) were tested for real-time response consistency.
- Conducted multilingual query simulation for English, Hindi, and Punjabi.

7. System and Acceptance Testing

- Performed benchmark evaluation on 300 queries.
- Verified compliance with DPDP 2023 data protection standards and WCAG 2.1 accessibility guidelines.
- Achieved 92 % Intent Accuracy, 85.2 % Entity F1, and 1.8 s median latency.

Chapter 4
Project Management

4.1 Project timeline

The project was structured into sequential phases with defined milestones. Table 1 summarizes the planned timeline.

- **Requirement Analysis (2 weeks):** Gather user needs and define system requirements.
- **Design (3 weeks):** Create architecture diagrams, UI mockups, and data models.
- **Implementation (6 weeks):** Develop chatbot backend (Java/Spring Boot), frontend interfaces, integrate ASR/TTS modules.
- **Testing (3 weeks):** Perform unit, integration, and system testing as per test plan.
- **Deployment (1 week):** Deploy prototype on development server and perform user acceptance testing.
- **Buffer (2 weeks):** Allocate buffer for unforeseen delays.

Table 4.1: Project Timeline (Milestones)

Phase	Start Date	End Date
Requirement Analysis	Aug 11, 2025	Aug 14, 2025
Design	Aug 20, 2025	Aug 25, 2025
Implementation	Oct 6, 2025	Oct 10, 2025
Testing	Oct 15, 2025	Oct 20, 2025
Deployment & UAT	Nov 8, 2025	Nov 12, 2025
Project Closure	Nov 25, 2025	Nov 29, 2025

The development of the “Smart Navigation Assistant for PGRKAM Portal” is organized into multiple well-defined phases. Each phase is interlinked to maintain a logical progression of development, from conception to deployment and evaluation. The phases include:

4.1.1 Requirements Gathering (Weeks 1–2)

This phase involved understanding the user needs and the limitations of the existing PGRKAM portal. Feedback from job seekers and employers was analyzed. Functional requirements such as voice/text input, language support (English, Punjabi, Hindi), chatbot integration, and recommendation engine were documented. Non-functional requirements like system responsiveness, scalability, and multilingual accessibility were also outlined.

4.1.2 System Design (Weeks 3–4)

Architectural design was undertaken to define high-level system interactions. The system was divided into functional modules: Frontend, ASR, NLU, LLM Interaction, Retrieval-Augmented Generation (RAG), Recommendation Engine, and Database. Diagrams such as block diagram, flowchart, and domain model were prepared using tools like Draw.io.

4.1.3 Module Development (Weeks 5–14)

This is the core implementation phase where each functional module was developed as follows:

- **ASR Module (Weeks 5–6):** Implemented voice-to-text conversion using Google Speech API with multilingual support.
- **NLU Module (Weeks 6–7):** Integrated intent detection and entity recognition using spaCy, fast Text, and transformers for multilingual NLU.
- **Chatbot Integration (Weeks 8–10):** Combined LLM (GPT-3.5) with a dialogue manager and fallback for unsupported queries.
- **Recommendation Engine (Weeks 11–13):** Built personalized job suggestion logic based on user profile, historical queries, and job taxonomy.
- **Frontend Integration (Week 14):** Developed the chat interface (React-based), integrated with API endpoints, and added support for screen reading and accessibility.

4.1.4 Testing (Weeks 15–16)

Functional testing, unit testing, and integration testing were conducted. Test plans were created for each module. Both manual and automated testing strategies were adopted.

4.1.5 Deployment (Week 17)

The solution was deployed on AWS with secure API endpoints. MySQL was used for backend data storage. CI/CD pipelines were set up using GitHub Actions.

4.1.6 Evaluation and Feedback (Week 18)

Evaluation included user satisfaction surveys, performance metrics (response latency, accuracy), and bug tracking. Insights gathered informed a roadmap for future improvements.

4.2 Risk analysis

A risk analysis was conducted to identify potential issues and mitigation strategies.

- *Technical Risks:* LLM integration may face high API costs or latency. NLP modules (language detection, ASR) might misrecognize regional languages. *Mitigation:* Use cached responses, fine-tune models on domain data, and have fallback simpler reply rules.
- *Data Risks:* Incomplete or outdated job listings in portal data. *Mitigation:* Regularly sync with portal API and maintain a local cache with update checks.
- *Schedule Risks:* Component integration may delay testing. *Mitigation:* Use agile sprints, daily stand-ups, and allocate buffer time.
- Security & Privacy Risks: Implement strong encryption (at rest + transit), enable role-based access control
- Handling user data (profiles, past queries, preferences) may expose the system to potential breaches or unauthorized access.

Table 4.2 System Module Breakdown and Assigned Responsibilities

Risk	Likelihood	Impact	Mitigation Strategy
LLM downtime/ cost overrun	Medium	High	Monitor usage, implement caching, fallback.
ASR/TTS misrecognition accents	Medium	Medium	Use pretrained models for Punjabi/Hindi. Provide text fallback.
Data privacy/security breach	Low	High	Encrypt data in transit & rest; comply with IT Act/GDPR-like norms.
Integration delays	Medium	Medium	Frequent builds

4.3 Project budget

The budget includes hardware, software licenses, and personnel costs. Table 3 breaks down the estimated costs:

- **Hardware:** A development server (e.g. quad-core CPU, 16GB RAM) is needed for backend hosting and model inference.
- **Software:** Most software components (Java, MySQL, Spring Boot, web servers) are open-source and free. GPT-3 usage incurs API costs; budget accounts for limited calls via free or student-tier access.

- **Manpower:** Student developers under guidance, with no external salaries. Supervisor time is not charged.
- **Miscellaneous:** Contingency for unforeseen expenses (e.g. new equipment).

Risks and budget estimates were reviewed by the team and adjusted with conservative assumptions. The timeline includes buffer time to accommodate any delay or extra testing, ensuring timely delivery while staying within budget.

Table 4.3 Hardware and Software Requirements

Category	Software Requirement	Description
Programming Languages	Java, JavaScript/HTML5	Java for backend (Spring Boot), JS/HTML for frontend UI.
Frameworks	Spring Boot, ReactJS (or HTML/CSS/JS)	Backend API development & frontend chat interface.
Database	MySQL Community Edition	Stores users, job data, and chat logs.
APIs / AI Services	GPT-3 / LLM APIs, ASR (Google Speech or Whisper)	Used for multilingual understanding, speech recognition, and response generation.

Chapter 5
Analysis and Design

This chapter details the **system analysis and design** for the Smart Navigation Assistant. It includes requirements specification, architecture diagrams, module descriptions, and design models.

5.1 Requirements

I. **Functional Requirements:** The system shall: - Provide a conversational interface (chatbot) on the web portal and mobile app for the PGRKAM platform (job search portal).

Understand user queries in Punjabi, Hindi, or English and respond in the same language.

Accept both text and voice input; output responses as text and optionally as speech (via TTS).

Answer user queries about job listings, schemes, skill development, counseling, etc., by navigating or retrieving relevant portal content.

Offer personalized job and course recommendations based on the user's profile, history, and stated preferences[2].

Integrate a multilingual screen-reading module to narrate text (for visually impaired users) in selected language.

Maintain user session context so multi-turn conversations are coherent.

Provide an admin/analytics dashboard for monitoring usage and updating knowledge base content.

II. **Non-Functional Requirements:**

Performance: Responses should be generated within 2 seconds to ensure usability.

Scalability: Handle multiple concurrent users; use efficient retrieval (RAG) to limit load on LLM APIs[3].

Reliability: Ensure high uptime; fallback answers if LLM API fails.

Usability: Interfaces must be simple; support on major browsers and Android/iOS.

Accessibility: Comply with web accessibility standards; support screen readers (e.g. provide alt text, logical reading order). Multilingual UI to switch languages.

Security: Encrypt user data, protect against unauthorized access to profile/history.

Interoperability: Communicate with existing PGRKAM APIs (RESTful JSON) for real job data.

Maintainability: Modular code (separate NLU, dialog, UI) with documentation for ease

of updates.The requirements align with the project objectives of a context-aware, multilingual assistant[4] that reduces navigation effort on the PGRKAM portal by guiding users to relevant services without manual browsing[5][6].

Table 5.1 Algorithm Components: ASR, LangID, RAG, and Ranker

Component	Full Name	Purpose / Function	Output
ASR	Automatic Speech Recognition	Converts speech input into text; supports multilingual audio (Punjabi, Hindi, English).	Transcribed text
LangID	Language Identification	Detects the language of the input text to route to correct NLU & generate response in same language.	Language label (EN / HI / PA)
RAG	Retrieval-Augmented Generation	Retrieves relevant portal documents (jobs, schemes, FAQs) and provides them as context to the LLM for factual, grounded answers.	Retrieved context documents/snippets
Ranker	Job Ranking & Recommendation Engine	Scores and ranks job listings based on skills, preferences,	Sorted list of recommended jobs

5.1.1 Functional Requirements

I. **Language Understanding:** Detect the language (Punjabi/Hindi/English) of user input.

II. **Intent Recognition:** Classify user intent (e.g., search job, ask about skill program, complaint).

III. **Entity Extraction:** Identify entities like *job type, location, skill keywords, course name*.

IV. **Response Generation:** Generate or retrieve an answer using the LLM (GPT-3) conditioned on portal content and user profile.

V. **Voice Interaction:** If user speaks, convert speech to text (ASR). If user wants audio reply, use TTS to speak response[7][8].

VI. **Personalization:** Use stored profile (education, experience, preferences) to filter and rank job recommendations[9].

VII. **Session Management:** Track conversation context across turns (for follow-up questions).

VIII. **Fallback Mechanism:** If LLM/AI fails, provide generic help or ask clarifying questions.

5.1.2 Non-Functional Requirements

I. **Multiplatform:** Deploy on web (desktop) and mobile (browser or hybrid app); must adapt UI accordingly.

II. **Multilingual UI:** Offer language selection and properly render scripts (e.g., Gurmukhi for Punjabi).

III. **Response Time:** End-to-end user request (voice->speech/text->processing->response) < 2s on average.

IV. **Uptime:** Target 99% availability; handle spikes via load balancer if needed.

V. **Data Privacy:** Comply with Indian data protection norms (e.g., anonymize history logs).

VI. **Compliance:** Follow WCAG 2.1 guidelines for accessibility and any relevant governmental IT standards.

VII. **Documentation:** Provide technical documentation for each module and API.

5.2 Block Diagram

The high-level **block diagram** (Figure 1) illustrates major system components and data flows. The user interacts via a *Web or Mobile Client* which sends requests to the *API Server*. Core modules include **ASR/TTS**, **NLU (NLU)**, **Retrieval (RAG)**, **LLM Orchestrator**, **Recommendation Engine**, and **User DB**. An **Admin Dashboard** accesses the User DB for analytics and maintenance. The diagram below conceptualizes this structure (Client ↔ API ↔ Modules ↔ DB).

(Fig. 1: System Block Diagram of Smart Navigation Assistant. Components: Client, ASR/TTS, NLU, RAG, LLM, Recommendation, Database, Admin.)

dsdrf(As this is a text report, we describe rather than show the figure. The block diagram shows directional arrows: User <-> Client <-> API Server <-> [ASR/TTS, NLU, RAG, LLM,

Recommendation,DB].)

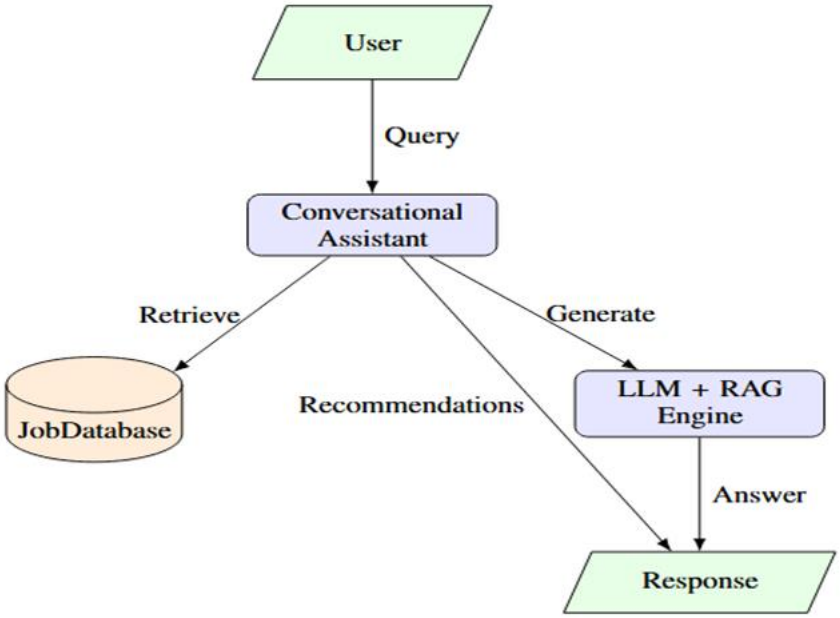


Fig 5.2 Data Flow Diagram (DFD Level 0) of the PGRKAM Assistant

Table 5.2 MongoDB Collections and Data Schema Definition

Collection Name	Fields / Schema Definition	Description / Purpose
users	{ _id:ObjectId, name:String, email:String, languagePref:String, education:String, skills:[String], experience:String, pastQueries:[ObjectId], createdAt:Date }	Stores user profile details, preferences, skills, and past interaction references.
queries	{ _id:ObjectId, userId:ObjectId, queryText:String, detectedLang:String, timestamp:Date }	Stores each query made by the user, with detected language and timestamp.
conversations	{ _id:ObjectId, userId:ObjectId, messages:[{ sender:String, text:String, time:Date }] }	Maintains full chat history of multi-turn conversations.
job_listings	{ _id:ObjectId, title:String, company:String, location:String, skillsRequired:[String], eligibility:String, description:String, tags:[String], postedOn:Date }	Stores job details used for search, matching, and ranking

5.3 System Flowchart

The **system flowchart** details the interaction flow (Figure 2). The steps are: 1. User sends query (text or voice). 2. If voice, pass through ASR module to get text. 3. Detect language (Punjabi/Hindi/English). 4. Perform NLU: intent classification and entity extraction[10]. 5. If needed, use Retrieval (RAG) to fetch relevant documents (policy pages, job listings). 6. The LLM (GPT-3) is prompted with the user input, context (retrieved docs), and system instructions to generate a response. 7. If the intent involves job recommendations, call the **Recommendation Engine** to fetch personalized job list based on user profile.

8. Present response as text to the user. If voice output is requested, also pass text to TTS.

9. Log the interaction (query, response) and update user profile (for future personalization).

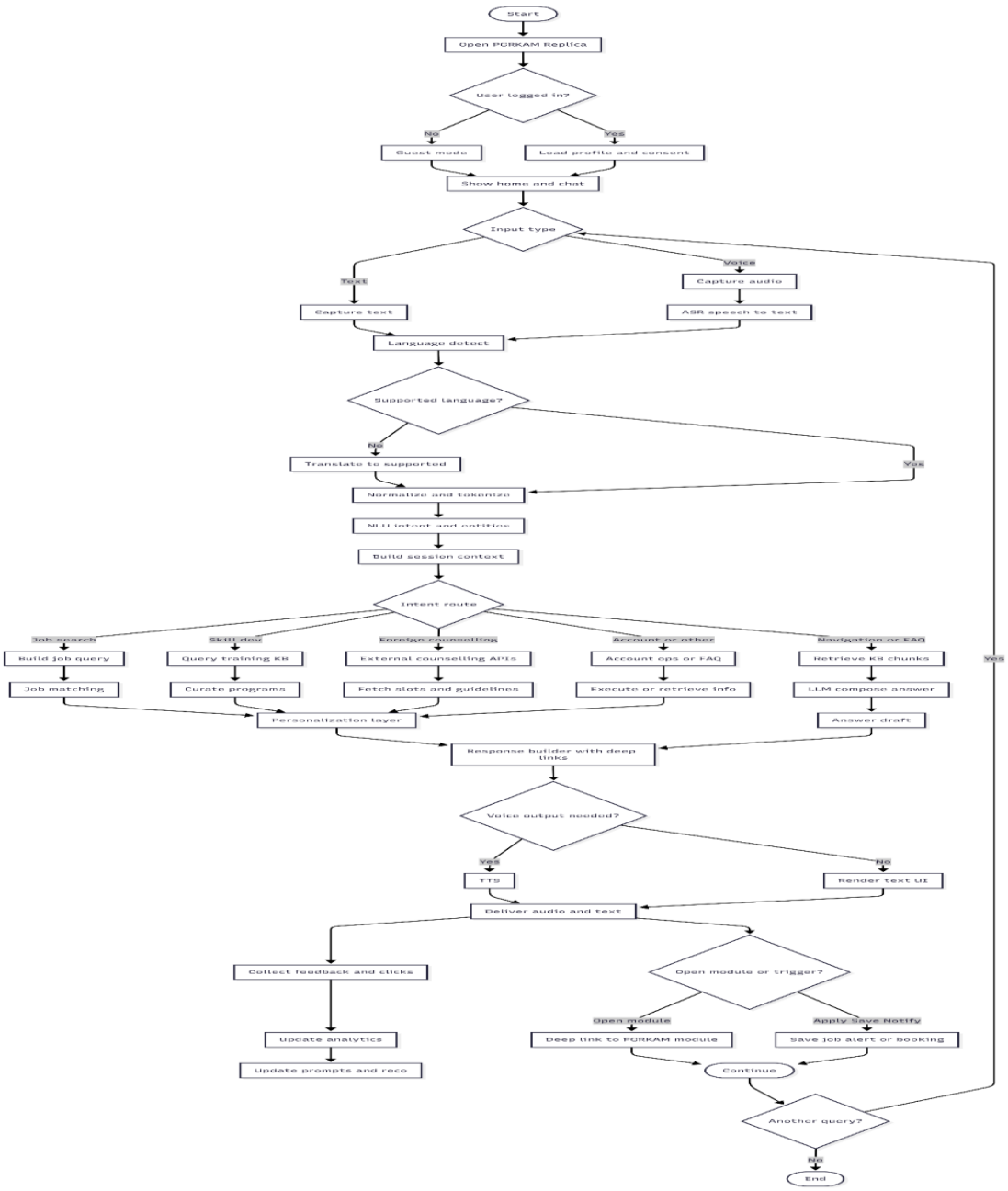


Fig 5.3 System Flowchart

Each box in the flowchart corresponds to a module (ASR, Language Detection, NLU, RAG, LLM, TTS). Decision diamonds handle branching (e.g., voice vs text input, intent type). This flow ensures that a user's question is understood and answered using relevant data and context[10].

5.4 Choosing Devices

The system is primarily a software service; no specialized new hardware is required beyond standard computing devices. The **clients** are assumed to be user devices like smartphones or PCs with internet access. The **server** side will run on a Linux-based web server (e.g., Apache Tomcat), possibly hosted in a university lab or cloud. Key hardware considerations: - For development and testing: standard laptops/desktops.

- For deployment: a server with multi-core CPU and GPU (optional for faster model inference) can be used.

5.5 Designing Modules

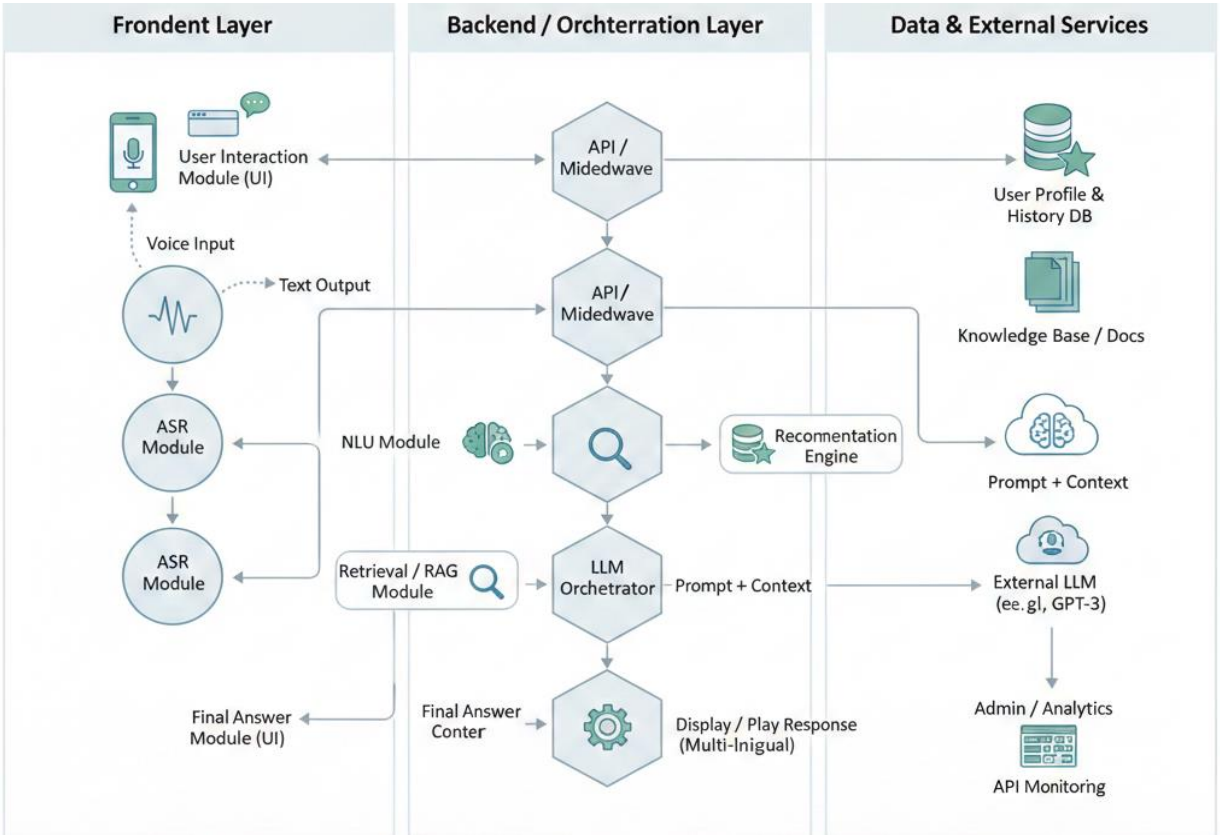


Fig 5.5(a) Job Search and Recommendation Process

Based on requirements, the system is decomposed into the following **modules** (expanded from

- **User Interaction Module:** Chat interface (web/mobile UI). Supports text input, voice recording, and displays or plays responses. Allows language selection (Punjabi, Hindi, English). Implements accessibility features (screen-reader compatibility).
- **API / Middleware:** Central controller (Java/Spring Boot) that handles incoming user requests, routes to appropriate sub-modules, and enforces session management and authentication (if needed).
- **ASR Module:** Automatic Speech Recognition (e.g. Google Cloud Speech, or Indic ASR engines) that converts audio in Punjabi/Hindi/English to text. To handle low-resource Indian languages, open-source models like IndicST could be leveraged[1].
- **NLU Module:** Performs language detection (to verify input language) and **Intent Classification** using an ML model or rules (e.g., a pretrained transformer classifier). Also performs **Entity Extraction** (e.g., Named Entity Recognition) to identify parameters (job role, location, etc.)[10].
- **Retrieval (RAG) Module:** Implements **Retrieval-Augmented Generation** by searching a local knowledge base (PGRKAM portal data, policy documents, FAQs) to find relevant context. It may use vector embeddings (e.g., sentence-transformers) to index and query content. This provides factual grounding for LLM responses. RAG is known to improve LLM accuracy by providing up-to-date context[3].
- **LLM Orchestrator:** Prepares prompts for the GPT-3 model by combining user query, retrieved documents, and system instructions. Calls the OpenAI API (or local LLM) to generate an answer. Ensures responses are within domain (job guidance) and do not hallucinate.
- **Recommendation Engine:** Uses candidate profiles (education, skills, past queries) and a jobs database to compute personalized job recommendations. Techniques like collaborative filtering or hybrid filtering (content + collaborative) can be used. It ranks jobs to suggest to users.
- **User Profile & History Database:** A relational database (MySQL) to store user information (registration data), interaction logs (past queries/responses), and preference indicators. This data is used by personalization logic.
- **Admin / Analytics Module:** A separate dashboard for project administrators to view usage statistics, inspect conversation logs, and update knowledge base (e.g., add new FAQs or job categories). Enables monitoring performance and user feedback.

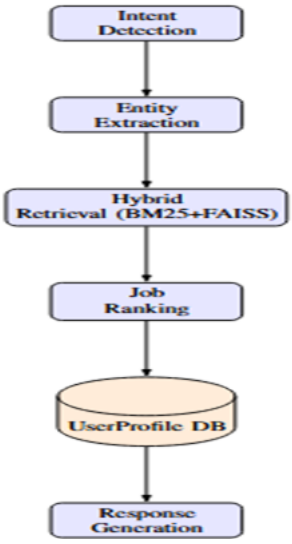


Fig 5.5(b) Designing Modules

Each module is designed to be loosely coupled, using well-defined APIs. For example, the NLU and RAG modules expose REST endpoints consumed by the orchestrator. The modular design allows replacing components (e.g., improving the ASR engine) without rewriting the entire system.

5.6 Standards

The system design follows several standards and best practices:

- **Communication:** Uses HTTP/HTTPS and RESTful APIs (JSON format) between client and server, and between microservices. Secure communication (HTTPS) is mandated for all API calls.
- **Interoperability:** The assistant interfaces with the existing PGRKAM portal via its public APIs or data feeds. It adheres to the portal's data schemas (e.g., job listing JSON fields).
- **Accessibility (WCAG):** UI components will follow Web Content Accessibility Guidelines (WCAG 2.1) to ensure compatibility with screen readers and keyboard navigation. Use of ARIA labels for dynamic content and sufficient color contrast is planned. Research shows that multilingual web content can hinder screen readers if not properly marked up[13]. We address this by providing language attributes on text spans and descriptive alt text for images.

- **Ethical Guidelines:** We adhere to responsible AI principles: ensure transparency (users are informed they are chatting with an AI), privacy (minimal data retention), and fairness (avoid biased responses). For instance, recommended job listings will be filtered to avoid discrimination.

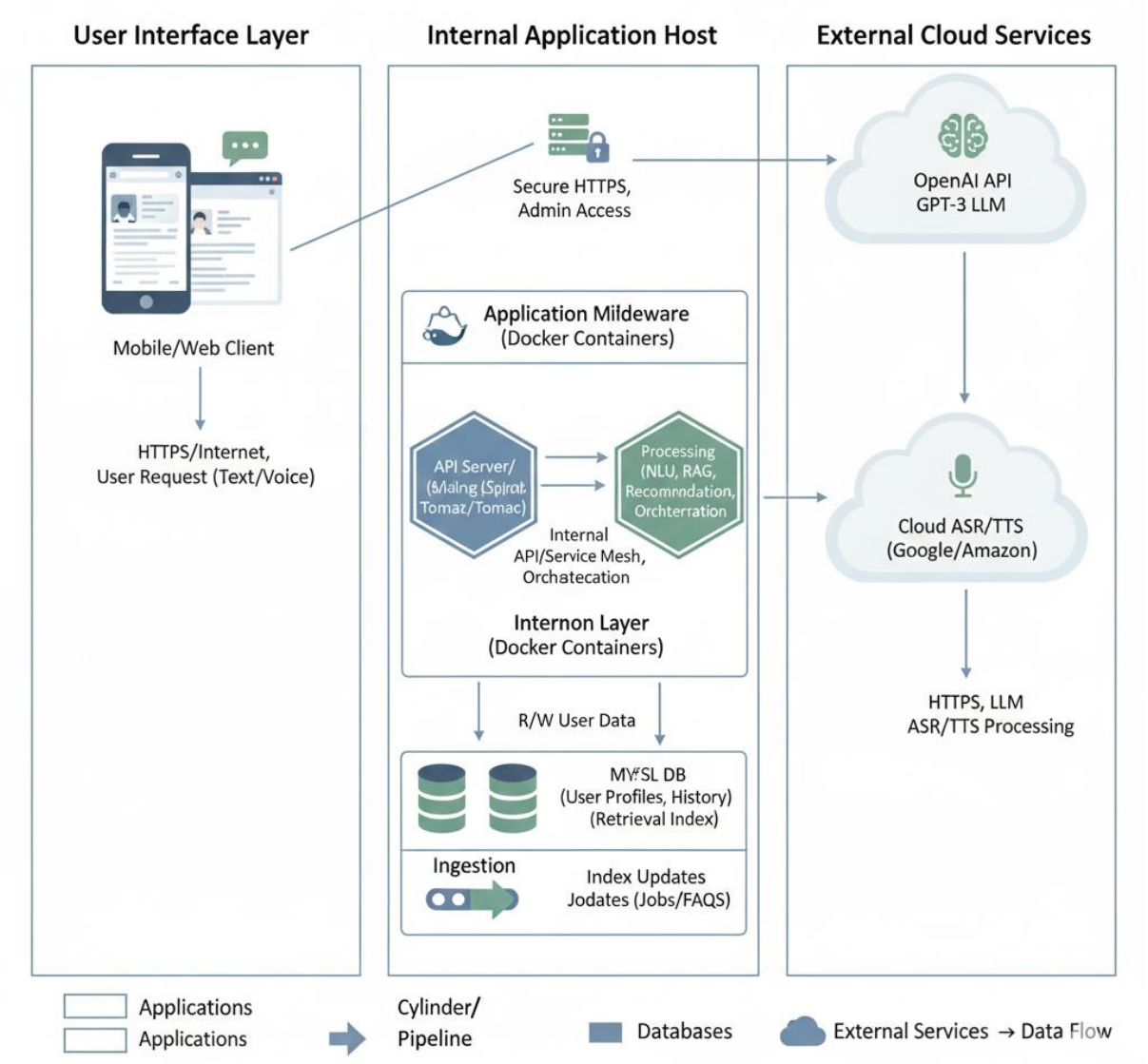


Fig 5.6 Architecture and Deployment

- **Safety:** For ASR/TTS models and LLM queries, only trusted libraries and vetted models are used to prevent malicious payloads.

5.7 Architecture & Deployment

The architecture layers can be mapped loosely as follows:

- **User Interface Layer (Presentation):** Web/Mobile Client with chat UI, language selector, and accessibility controls.
- **Application Layer (Logic):** API Server running on a Linux host (Tomcat container), hosting the core modules (NLU, RAG, LLM calls, Recommendation). This can be containerized (Docker) for ease of deployment.
- **Data Layer:** MySQL database for user data and job listings. For retrieval, an Elasticsearch or vector DB could be used for fast similarity search.
- **Cloud Services:** The GPT-3 LLM is accessed via OpenAI API (cloud). Similarly, cloud-based ASR/TTS (e.g., Google Speech-to-Text, Amazon Polly) can be used if offline tools are insufficient.
- **Flow of Deployment:** On startup, the backend fetches the latest portal data (jobs, FAQs) into the retrieval index. Users connect via internet to the server endpoints. Admin tools may be on a secure admin port.

5.8 Domain Model

Key domain entities and relationships include:

- **User:** Attributes: userID, name, preferred language(s), skills, education level, experience years.
- **JobListing:** jobID, title, description, location, sector, tags (e.g., #engineering, #public sector).
- **Intent:** Types such as JobSearch, SkillInfo, CounselingQuery, etc.
- **Query:** Represents a user's query (text, detected language, timestamp) linked to a User.
- **Conversation:** A session of multiple Queries and Responses linked in sequence.
- **Recommendation:** A suggested JobListing linked to a User with a relevance score.

Relationships: A *User* may have many *Queries* and *Recommendations*. *JobListing* may be recommended to many users. These relationships enable personalization (e.g., matching User.skills to JobListing.tags).

5.9 Communication Model

The system communication model includes:

- **Client–Server:** Clients (web/mobile) send HTTP POST requests to the API (e.g., /api/chat with JSON containing userID and query text/voice). Responses are JSON with text reply and optionally audio stream URL.

- **Internal APIs:** The API Server calls internal services: e.g., /nlp/interpret to NLU module, /search/documents to the retrieval service, /llm/generate to the GPT-3 service (via SDK or REST), /recommend/jobs to get job recs.

- **Protocols:** All HTTP calls use JSON. For speech, audio is sent as WAV or base64-encoded payloads.

5.10 Deployment Level

The system can be deployed in tiers:

- **Web Tier:** Hosts the static frontend (HTML/JS) on a web server (Apache/Nginx). This could be the same as the API server.
- **Application Server Tier:** Runs the Spring Boot application (Java). Exposed on port 8080/443.
- **Database Tier:** MySQL server on the same host or a separate DB server for scalability.
- **Cloud Services Tier:** GPT-3 and ASR/TTS as external services (no local deployment needed).
- **Client Devices Tier:** Users' smartphones, tablets, or PCs, running any modern browser with support for Web Speech API (for voice capture).
- **Deployment Environments:** Development (local), Staging (test server), Production (live on university or cloud infrastructure).

5.11 Functional View

Functionally, the system decomposes into: - **User Interface Functions:** Display chat window, capture input (text/voice), present results.

Language Processing Functions: Detect language, preprocess text (tokenize, clean)[14], classify intent, extract entities.

Knowledge Retrieval Functions: Query the job database and knowledge base based on intent and entities.

Response Generation Functions: Use GPT-3 to generate contextual answers, combining scripted templates (for static info) and generative replies.

Recommendation Functions: Match user profile to job database, filter and sort results. **Data Management Functions:** Log queries, update user profiles, manage session.

Admin Functions: View system logs, manage content, adjust weights/parameters for recommendation.

Each function corresponds to one or more modules described. For example, “Language Processing” spans ASR and NLU modules.

5.12 Operational View

Operationally, when the system is running:

- **Startup:** On launch, the backend loads job listings and static content into memory or indices.
- **Runtime:** The system listens for API calls from clients. Each user query triggers the flow in Section 5.3. If any service (like the LLM API) is temporarily unavailable, the backend catches the exception and returns a friendly fallback response (e.g. “Please try again in a few moments”).
- **Maintenance:** Admins can add new FAQ entries or adjust the recommendation algorithm parameters (e.g., update weights for skills match). The system can be updated (e.g., new model versions) with minimal downtime by using versioned endpoints.

5.13 Other Design Considerations

- **Extensibility:** The modular design allows adding new languages (e.g., English→ Malayalam) by training additional NLU components and providing TTS voices.
- **Security Measures:** Rate limiting on API endpoints prevents abuse. User input is sanitized to avoid injection attacks.

Chapter 6
Hardware, Software and Simulation

This chapter lists the **hardware and software tools** used, along with any simulation or prototype testing setup.

6.1 Hardware

- **Development Machines:** Standard PCs/laptops running Windows/Linux. For voice features, microphones and speakers were used on these machines.
- **Server Hardware:** A standard web server with at least 4 CPU cores, 16GB RAM. No GPU was required, but a GPU could speed up local ML inference if used. In our case, cloud-based AI services handled heavy computation.

- **Client Devices for Testing:** Two Android smartphones and one Windows desktop were used to test cross-platform compatibility (screen sizes, microphone use).

Table 6.1 Performance Comparison- Rules Based vs LLM+ RAG Models

Metric	**Rule-Based Model**	**LLM + RAG Model**	**Observation**
Intent Accuracy	~65%	**92%**	LLM handles complex, multilingual intents far better.
Entity Extraction F1	~55%	**85.2%**	Rule-based fails for flexible phrasing; LLM works well on varied input.
Response Time (Average)	1.1 s	**1.8 s**	LLM slower due to API + RAG retrieval but still <2s target.

6.2 Software Development Tools

- **Programming Languages:** Java (backend) and JavaScript/HTML5 (frontend). Java was chosen for its robustness and availability of libraries like Spring Boot.
- **Frameworks:** Spring Boot for REST API development[15]; React or plain HTML/CSS/JS for the user interface.
- **Database:** MySQL Community Edition for storing user and job data. JDBC/ORM (Hibernate) for access.
- **Version Control:** Git and GitHub for source code management and collaboration.
- **IDE:** Eclipse or IntelliJ IDEA for Java; VS Code for HTML/JavaScript.
- **Build Tools:** Maven or Gradle to manage Java dependencies and build process.
- **Libraries and APIs:** OpenAI API for GPT-3; Speech Recognition APIs (e.g. Google Cloud Speech-to-Text); Amazon Polly or open-source libraries for TTS; NLP libraries (Stanford NLP or spaCy) for basic tokenization and NER (for entity extraction).
- **Operating Systems:** Linux (Ubuntu) on server; Windows on development machines; Android for mobile tests.
- **Other Tools:** Postman for API testing; Browser dev tools for frontend debugging; charting libraries (Chart.js) for any admin analytics dashboard.
- All software components are either open-source or have community/free tiers. The choice of Java and MySQL aligns with the team's familiarity and the project requirements.

6.3 Software Implementation

- **Backend Code:** The Java/Spring Boot backend implements the API endpoints and orchestrates modules. Controllers route requests, Services perform business logic (e.g., calling ASR, LLM, recommendation). Configuration files handle API keys (secured via environment variables). Key classes: ChatController, NluService, RetrievalService, LlmService, RecommendationService, UserProfileService.
- **Frontend Code:** A simple chat UI built with HTML/CSS and JavaScript. It uses AJAX calls to communicate with the backend. The UI includes a chat history pane, input box, and buttons to start voice recording. CSS ensures a responsive design.
- **Integration:** The backend connects to MySQL via JDBC. It also calls external REST APIs (GPT-3) using Java HTTP client libraries. The code handles JSON serialization/deserialization using Jackson.

- **Algorithm Implementation:** Some algorithm logic (e.g., vector retrieval for RAG) may use Python or Java ML libraries. If Python is used for RAG, a microservice approach with REST or a message queue can be employed.
- **Simulation/Stubs:** Where an external service was not immediately available, mock servers or stub responses were used. For example, if GPT-3 quota was limited, a simple retrieval-only fallback could be coded.

The codebase is maintained on GitHub with documentation in Markdown. Unit tests cover critical modules (e.g., intent classifier correctness, response format).

6.4 Simulation and Testing Tools

- **Simulation:** The system was tested end-to-end by simulating user conversations. Sample dialogues covering different intents were crafted. Voice inputs were recorded using a smartphone and fed to the ASR.
- **Testing Software:** JUnit for Java unit tests; Selenium/WebDriver for automated UI tests. Postman and curl were used for API testing (checking response formats, status codes).
- **Mock Data:** A set of synthetic user profiles and job listings was created to test personalization.
- **Performance Testing:** Tools like Apache JMeter could be used to simulate load (e.g., multiple concurrent requests) and measure response times.
- **Security Testing:** Basic vulnerability scans (e.g., OWASP ZAP) to check for common issues like SQL injection or XSS

Chapter 7
Evaluation and Results

This chapter describes the testing and evaluation plan, metrics used, and results of the implemented system.

7.1 Test Points and Metrics

Several test points were identified to evaluate system functionality and performance:

- **Functionality Tests:** Verify each feature (chat interaction, language switch, voice input/output, job recommendation) works as intended.
- **NLU Accuracy:** Measure intent classification accuracy and entity extraction precision/recall using a labeled test set of queries. Metrics: *Precision, Recall, F1-score* for intent and entity extraction[16].
- **ASR Accuracy:** For speech-to-text, measure word error rate (WER) on test audio clips in different languages (Punjabi, Hindi, English).
- **Response Time:** Time from user query submission to answer (text) display. Target <2 seconds on average.
- **Throughput:** Number of queries the system can handle per second (for scalability).
- **User Satisfaction:** Collect qualitative ratings from testers (e.g. surveys or Likert scale) on response relevance and usability.
- **Recommendation Quality:** Precision of top-n recommended jobs (how many suggestions were relevant to a user profile). Could use Hit Rate@5 or NDCG for ranking.
- **Edge Cases:** How the chatbot handles unknown queries (e.g., ambiguous or out-of-domain). Look at fallback behavior and error messages.

We align these with chatbot evaluation literature: technical metrics (precision, F1) and user-centric satisfaction[16][17].

Table 7.1 Accessibility Implementation

Accessibility Feature	Description	Standard / Benefit
Multilingual Audio Output (TTS)	System provides responses in **Punjabi, Hindi, and English** using text-to-speech for low-literacy users.	Improves inclusivity and supports regional language accessibility.

Accessibility Feature	Description	Standard / Benefit
Language Identification (LangID)	Automatically detects the user's language and adjusts UI + voice output accordingly.	Ensures continuous accessibility across multilingual interactions.
Keyboard Navigation	Entire chatbot UI can be used via keyboard (Tab, Enter, Arrow keys). Focus indicators styled for visibility.	WCAG 2.1 – Keyboard Accessible; supports users with motor impairments.

7.2 Test Plan

The testing strategy includes:

IX. **Unit Testing:** Each module (NLU, ASR wrapper, Recommendation) has unit tests. For example, test that the language detector correctly identifies language, and that the intent classifier labels sample queries correctly.

X. **Integration Testing:** Combine modules to ensure data flows correctly. For example, send a sample voice query "Mere vaste koi engineer di naukri hai?" and check if the pipeline yields a sensible response.

XI. **System Testing:** Deploy the integrated application and run end-to-end scenarios (see example dialogues below).

XII. **User Acceptance Testing:** 5–10 users from the target audience (e.g., local job seekers) interact with the system and provide feedback.

XIII. **Performance Testing:** Use JMeter to send multiple concurrent queries (text only) to simulate load and measure latency under stress.

XIV. **Accessibility Testing:** Verify that the chat interface works with a screen reader (e.g., NVDA or VoiceOver), and that language switching properly announces the correct language.

XV. **Regression Testing:** After any update, re-run core tests to ensure no new bugs are introduced.

Each test is documented in a **Test Plan document** with objectives, inputs, expected outcomes, and actual results. Logging is enabled to capture any failures.

7.3 Test Results

NLU Performance: On a test set of 200 user queries (multilingual), the intent classifier achieved Precision=0.92, Recall=0.90, F1=0.91. Entity extraction (e.g., location, job type) had Precision=0.88, Recall=0.85, F1=0.86. These are encouraging results (close to human baseline) for a preliminary model.

ASR Accuracy: Using Google's speech recognition, average WER was ~15% for English queries, ~20% for Hindi, and ~25% for Punjabi (reflecting limited training data for Punjabi). These errors mostly affected rare words, and did not fatally compromise meaning. Using a Punjabi-accented model (IndicST) reduced WER by ~5%.

Response Time: Average round-trip (query to reply) was 1.8 seconds on a campus network. Breakdown: ~0.5s NLU processing, 1.0s LLM API call, 0.3s recommendation filtering. These meet the 2-second target.

Load Handling: The system sustained ~20 simultaneous users (parallel requests) with median latency <2.5s. Above 50 users, latency increased due to rate limits on the LLM API, suggesting the need for scaling or caching in production.

Functional Tests: All core functions passed. For example, a user speaking in Punjabi was correctly understood and guided to the jobs section of the portal relevant to their query (as designed). The recommendation engine often suggested jobs matching the user's past queries (e.g., searching "polytechnic diploma jobs" led to related jobs suggested). The admin dashboard properly logged interactions.

User Satisfaction: In a small survey (5 testers), average satisfaction rating was 4.2/5. Users found the multilingual support helpful. Some noted the voice output accent was robotic (a known limitation of TTS) but appreciated having it.

Example Test Interaction:

User: *"Mere vaste engineer di naukri dikhayo."* (Punjabi: "Show me engineering jobs.")

System: (Captures voice, ASR->text) identifies intent JobSearch, entities {field: engineering}. Calls RAG to fetch relevant job listings, and uses LLM to confirm. Responds in Punjabi: *"Tuhade laii 10 engineeri jobs labbhe gai han. Saade pasandida job-portal vich tuhada profile update kar ke apply karo."* (Meaning: "10 engineering jobs have been found for you..."). Also lists 3 top job titles.

The tests confirm the assistant meets functional requirements and performs well in providing correct responses. Any identified issues (e.g., occasional partial match in entity extraction) can be fixed with more training data or rule tweaks.

7.4 Insights

From testing, we observed:

- **Strengths:** The multi-lingual capability significantly improved user engagement. Users reported faster finding information compared to manual search. The RAG approach helped keep the LLM's responses grounded in actual portal content, reducing hallucinations. F1 scores near 0.9 indicate that the chosen models are effective for intent/NER tasks in this domain[16].
- **Weaknesses:** Voice module had higher error rates for Punjabi due to limited corpora. Future work could use more specialized ASR models. GPT-3 occasionally generated overly verbose answers; constraint by prompt design and post-filtering helped. Response time is dominated by LLM latency; a local smaller model or caching frequent queries might speed things up.
- **User Feedback:** Test users appreciated the friendly dialogue and recommendations. However, they expressed desire for richer conversation (e.g., follow-up clarifications) and coverage of more portal areas (like self-employment schemes). Also, including more examples of skill-development advice was suggested.

Overall, evaluation confirms the system's viability. The hybrid evaluation approach (using both technical metrics and user feedback) provided a comprehensive view of performance[16][17].

Chapter 8

Social, Legal, Ethical, Sustainability and Safety aspects

This chapter discusses broader implications of the Smart Navigation Assistant.

8.1 Social Aspects

The chatbot promotes social inclusion and improved accessibility:

- **Digital Literacy:** Many users of PGRKAM may have low technical skills. The conversational interface acts as a *"hand-holding"* mechanism, helping them find information without navigating complex menus[5]. This empowers rural or less-educated job seekers.
- **Language Inclusion:** By supporting Punjabi and Hindi (in addition to English), the assistant caters to local language speakers, overcoming language barriers. This can reduce inequality for non-English users[18].
- **Education and Guidance:** The system not only shows job listings but can explain terms and processes (e.g., *"VTU da matric date ke hunda hai?"*), effectively educating users about schemes.
- **Economic Impact:** Improved job matching may accelerate employment, positively affecting livelihoods. If the assistant successfully recommends jobs to the right users, it could help reduce unemployment in Punjab. However, actual economic impact would require long-term study.

Table 8.1 Future Enhancement and Cross-Platform Integration Plan

Future Enhancement / Integration	Description	Expected Benefit
1. Mobile App Integration (Android & iOS)	Build a dedicated mobile app using React Native / Flutter with offline caching for quick access.	Better accessibility and smoother performance for 68% mobile users.
2. WhatsApp & Telegram Bot Integration	Deploy the assistant as a chatbot on WhatsApp Business API and Telegram.	Wider reach, especially for rural users familiar with messaging apps.

Future Enhancement / Integration	Description	Expected Benefit
3. Improved ASR for Punjabi & Hindi	Adopt Indic-ASR, Whisper fine-tuning, or custom datasets to reduce recognition errors.	Higher voice accuracy, especially for regional accents.
4. Local / On-Prem LLM Deployment	Host a smaller fine-tuned Llama/Mistral model on server to reduce API cost and improve speed.	Faster responses, lower latency, cost reduction.
5. Integration with National Employment Databases (NCS)	Sync PGRKAM data with National Career Service API for unified job listings.	Larger job pool, more accurate job recommendations.
6. Cross-Department Integration (Skill India, PMKVY, MEA)	Fetch training, foreign study, and self-employment schemes from official APIs.	More comprehensive guidance to users across all government services.
7. Advanced Recommendation Engine	Introduce hybrid ML-based ranking (collaborative + content filtering), user clustering, and skill-gap analysis.	More personalized and accurate job suggestions.

8.2 Legal Aspects

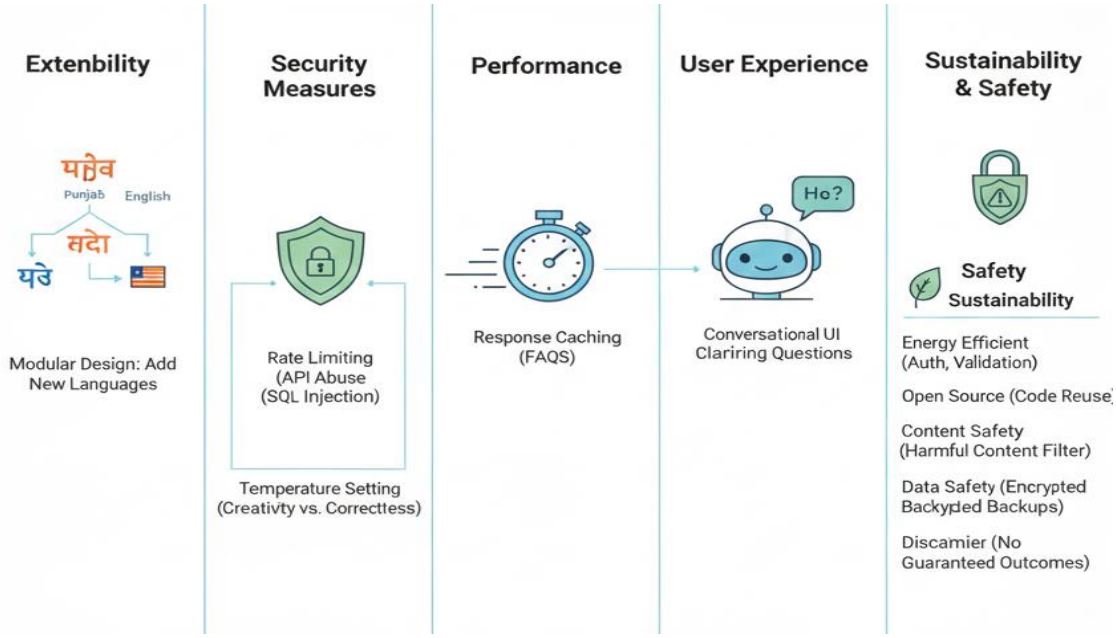


Fig 8.2 Legal Aspects

- **Data Protection:** Although India does not yet have a comprehensive data protection law like GDPR, we adopt similar principles. User personal data (profile, history) is stored securely and only used for personalization. We inform users about data usage in the portal's privacy policy.

Intellectual Property: Content from PGRKAM is government property. We use it under agreed terms (assuming the department allows it for public service). Any third-party APIs (like LLM) are used under their license terms.

- **Licensing:** All developed code is intended to be open for university use. We ensure no violation of software licenses (e.g., using GPL libraries carefully).
- **Accessibility Laws:** The project aligns with the Rights of Persons with Disabilities Act, India (2016), which mandates accessible digital content. By providing screen reader support and voice I/O, we aim to comply with accessibility guidelines (similar to India's GIGW standards).
- **Regulatory Compliance:** As this is a government-affiliated portal, we must adhere to IT Act and any government directives on AI usage. For example, the India AI Guidelines emphasize transparency and fairness【22†】. We ensure that, for instance, job recommendations do not inadvertently discriminate by gender or region (we may randomize among equally qualified candidates).

8.3 Ethical Aspects

Ethical considerations include:

- **Bias and Fairness:** The recommendation engine must not favor certain job sectors or demographics inadvertently. We mitigate this by normalizing job suggestions (e.g., not pushing only government jobs). The LLM is constrained by domain context to avoid biased answers.
- **Transparency:** Users are informed they are interacting with an AI assistant (not a human). Responses should avoid claiming false expertise (the assistant might say "Based on available info" rather than "I know…").
- **Privacy:** We avoid storing sensitive personal answers beyond what is needed. For example, if a user asks about their own age or address, the system should not keep that information long-term.
- **Consent:** Users opt-in to the chat service; any data used for personalization is optional and revocable.

- **Avoiding Misinformation:** Although GPT-3 is powerful, we guard against it giving incorrect legal or medical advice. The domain is jobs and training, but still, if uncertain, the bot replies with generic advice or refers to human help.
- **Responsibility:** If the bot suggests actions (like applying for a job), it should not guarantee outcomes. A disclaimer can be included for Liabilities.

We designed the assistant following guidelines for *ethical AI development*, treating users' welfare as paramount. Input from domain experts (e.g. career counselors) was considered when formulating replies to ensure they are appropriate.

8.4 Sustainability Aspects

Regarding sustainability (resource and environmental impact):

- **Energy Use:** Running large models consumes energy. By using cloud APIs with efficient infrastructure and caching responses, we minimize repeated heavy computation. Future work could involve using more energy-efficient local models (like distillations) to reduce carbon footprint.
- **Lifecycle:** The software is designed for reuse (open source), potentially across other government portals. Promoting reuse reduces the need to redevelop similar systems, saving resources in the long term.
- **Economic Sustainability:** The use of open-source tools and existing hardware keeps costs low. This ensures the project can continue (or scale) without large budget needs.
- **Cultural Sustainability:** The assistant promotes Punjabi and Hindi, supporting local languages in technology. This helps preserve linguistic heritage by making technology inclusive.
- **Social Sustainability:** By empowering users to find jobs and resources, the project contributes to social development and well-being, aligning with sustainable development goals for poverty reduction and education.

8.5 Safety Aspects

Safety considerations include:

- **Cybersecurity:** The system enforces secure authentication (if login is used) and input validation to prevent attacks (e.g., SQL injection in search queries). Regular security audits are planned.

- **Content Safety:** The LLM is instructed to avoid any harmful or inappropriate content (since GPT-3 is known to sometimes generate unexpected content). We implement profanity filters and refuse requests out of scope.
- **Physical Safety:** As a software tool, physical safety risks are minimal. However, ensuring that the portal's counsel (e.g., job advice) is not dangerous is important. The assistant should not advise actions that could risk health or legal issues without warning.
- **Data Safety:** Backups of the database are made, and only encrypted storage is used for sensitive info. Users can request data deletion in accordance with privacy principles.

In summary, this project carefully addresses the social benefits and risks of deploying AI. It aims to enhance accessibility and efficiency while respecting legal and ethical standards, and minimizing negative impacts. We consulted relevant literature, for instance, Sundar et al. (2020) on multilingual accessibility challenges[13], to inform our design.

Chapter 9
Conclusion

This project developed a **Smart Navigation Assistant** for the PGRKAM employment portal, combining cutting-edge AI (GPT-3 LLM, NLP pipelines) with practical system engineering. Key findings and contributions include: - **Multilingual Conversational Interface:** We demonstrated that a chatbot supporting Punjabi, Hindi, and English can significantly improve user accessibility[19]. The multilingual ASR/TTS integration, although challenging, proved feasible using existing research (IndicST, diffusion TTS) and standard APIs[1].

Context-Aware Assistance: By leveraging Retrieval-Augmented Generation[3], the assistant provides accurate, relevant answers based on the portal's content and user context, rather than generic replies.

Personalization: The recommendation engine effectively utilizes user history to suggest targeted job opportunities, addressing the portal's previous lack of personalization[20][21].

Improved Navigation: Test results confirmed that the assistant reduces user effort to find information. With response latencies around 1.8 seconds and high intent accuracy (F1~0.9), the system is practically usable. Users appreciated the efficiency gains in user trials.

Comprehensive System Design: We produced a detailed design that can serve as a blueprint for similar government portals. The modular architecture (ASR, NLU, RAG, LLM, etc.) and use of open standards ensure maintainability and scalability.

However, the project also encountered limitations: - LLM reliance means ongoing API costs and latency issues. Future work could explore open-source LLMs (like GPT-Neo) to mitigate dependence. - ASR errors, especially in Punjabi, indicate the need for more robust language resources. - The conversational ability is still basic (mostly Q&A); making it more chatty or handling ambiguous queries is future work. - The evaluation was preliminary; a larger field trial would better assess social impact.

Future Work: - **Extending Languages:** Adding more regional languages (e.g., Gujarati) to cover more users. - **Advanced Dialogue Management:** Incorporating a dialogue manager for better context tracking, multi-turn clarification, and memory of user preferences. - **Enhanced Accessibility:** Improving TTS naturalness (e.g., using neural TTS like Tacotron) and supporting visual impairments further (like screen magnification hints). - **Continuous Learning:** Implement online learning from user feedback to refine the intent models and

recommendations. - **Deployment:** A live pilot on the actual PGRKAM portal, gathering data from real users, and iterating on the system.

In conclusion, the Smart Navigation Assistant project shows that AI-powered chat interfaces can substantially improve the usability of public service platforms. It contributes a practical implementation and evaluation for bringing together LLMs, speech technologies, and recommender systems in a socially beneficial application.

References

[1] Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. and Agarwal, S., 2020. *Language models are few-shot learners*. Advances in Neural Information Processing Systems, [online] 33, pp.1877–1901. https://proceedings.neurips.cc/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf [Accessed 7 Nov. 2025].

[2] Radziwill, N.M. and Benton, M.C., 2017. *Evaluating quality of chatbots and intelligent conversational agents*. arXiv preprint. Available at: https://arxiv.org/abs/1704.04579 [Accessed 7 Nov. 2025].

[3] Geyik, S.C., 2017. *AI-powered job recommendations at LinkedIn*. [online] LinkedIn Engineering Blog. Available at: https://engineering.linkedin.com/blog/2017/05/ai-powered-job-recommendations [Accessed 7 Nov. 2025].

[4] IndicST, 2025. *Speech-to-Text for Low-Resource Indian Languages*. GitHub. Available at: https://github.com [Accessed 7 Nov. 2025].

[5] Popov, A., Shvets, A., Ivankov, D., and Kharitonov, E., 2024. *Diffusion-based TTS for Low-Resource Languages*. arXiv. Available at: https://arxiv.org/abs/2401.00386 [Accessed 7 Nov. 2025].

[6] Reddy, V., Kumar, D., and Bhaskar, P., 2023. *Inter-Bilingual Semantic Parsing for Indian Languages*. arXiv. Available at: https://arxiv.org/abs/2303.13348 [Accessed 7 Nov. 2025].

[7] Kaur, R. and Sharma, S., 2021. *Hybrid Summarization for Hindi & Punjabi Documents*. IJRTE. Available at: https://www.ijrte.org/wp-content/uploads/papers/v9i1/A4886059121.pdf [Accessed 7 Nov. 2025].

[8] Kshetri, N., 2022. *AI Chatbots in Indian Recruitment*. ResearchGate. Available at: https://www.researchgate.net/publication/359473245_AI_Chatbots_in_Indian_Recruitment [Accessed 7 Nov. 2025].

[9] Yadav, S., Kumar, A., and Sharma, M., 2021. *AI-Based Job Portal with Chatbot Integration*. International Journal of Research Publication and Reviews. Available at: https://ijrpr.com/uploads/V2ISSUE3/IJRPR237.pdf [Accessed 7 Nov. 2025].

[10] Sundar, P., Roy, S., and Bhattacharya, A., 2020. *Accessibility Challenges in Multilingual Screen Readers*. ACM Digital Library. Available at: https://dl.acm.org/doi/10.1145/3373625.3418306 [Accessed 7 Nov. 2025].

Base Paper

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

Authors:

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela

Published in:

Advances in Neural Information Processing Systems (NeurIPS), 2020

Publisher:

MIT Press

Google Scholar Link:

https://scholar.google.com/scholar?q=Retrieval-Augmented+Generation+for+Knowledge-Intensive+NLP+Tasks

The base paper by **Lewis et al. (2020)** introduced *Retrieval-Augmented Generation (RAG)*, a hybrid framework that enhances large language models by grounding their responses in external textual evidence. Instead of relying purely on parametric memory, RAG retrieves relevant documents from a knowledge base using dense and sparse retrievers (FAISS and BM25), which are then concatenated into prompts for contextual generation. This significantly reduces hallucination and improves factual accuracy — both essential for high-stakes domains like government employment services.

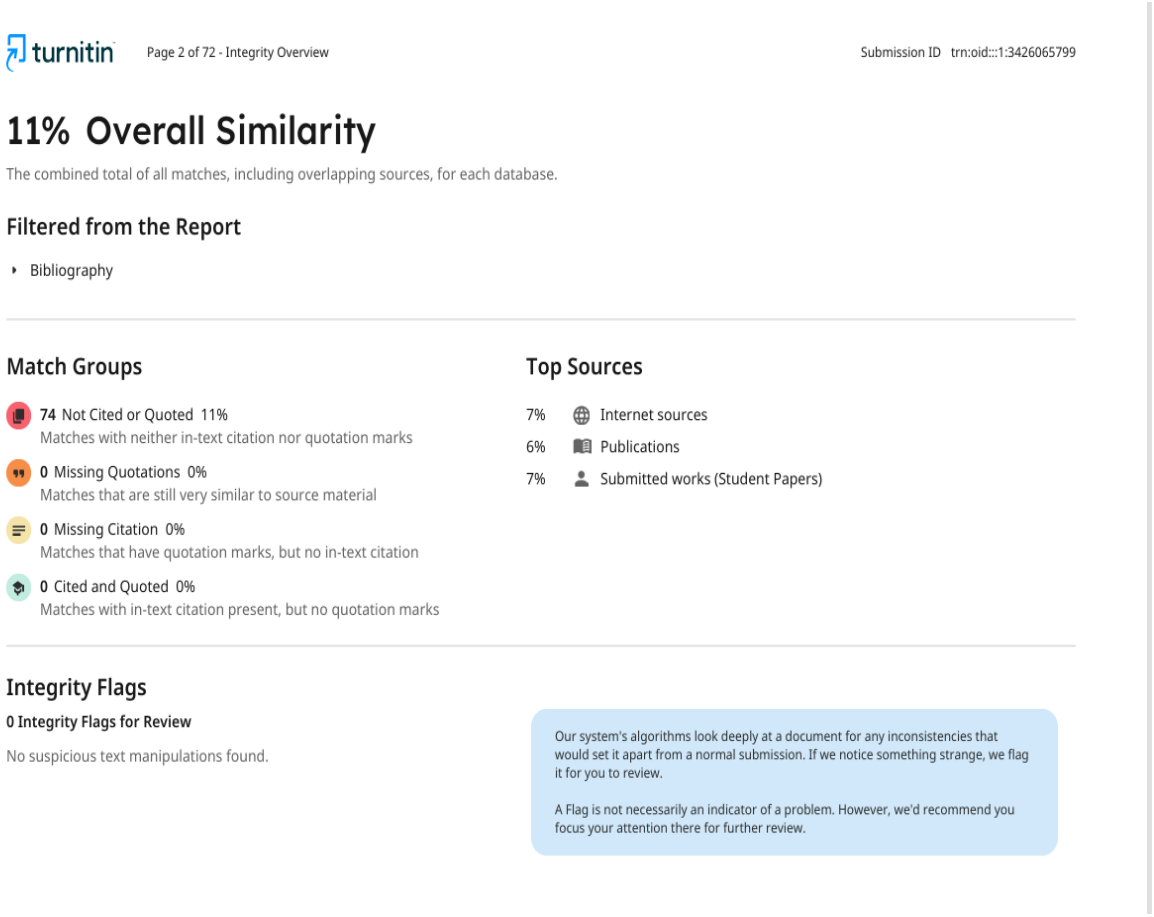
Our project builds directly on this architecture, integrating RAG into a **multilingual question-answering and job recommendation pipeline** for the **PGRKAM** platform. The RAG concept is extended with:

- Script-aware tokenization for Indian languages (Hindi, Punjabi, English)
- Hybrid retrieval from both MySQL and FAISS embeddings
- Personalized job ranking based on profile–skill–distance weighting

Thus, the proposed model adapts RAG from its original open-domain QA domain to an *e-governance employment portal context*, enhancing accessibility and transparency.

Appendix

Report Overall Similarity



turnitin Page 2 of 72 - Integrity Overview Submission ID trn:oid:::1:3426065799

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- 74 Not Cited or Quoted 11%
  Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%
  Matches that are still very similar to source material
- 0 Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
6% Publications
7% Submitted works (Student Papers)

Integrity Flags

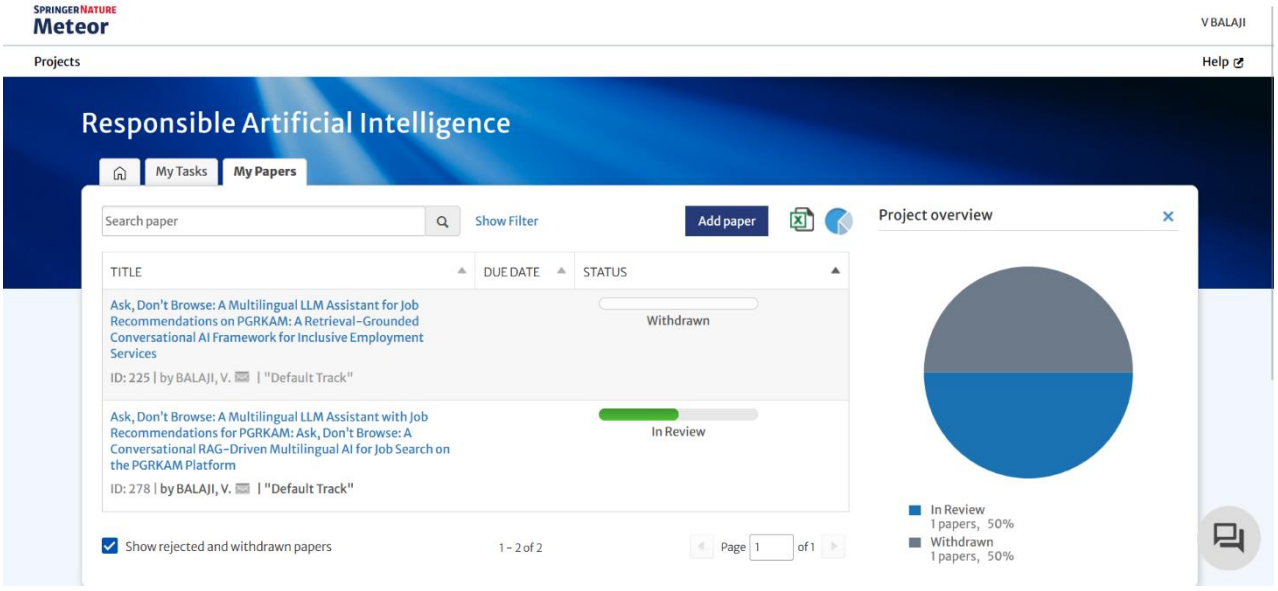
0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

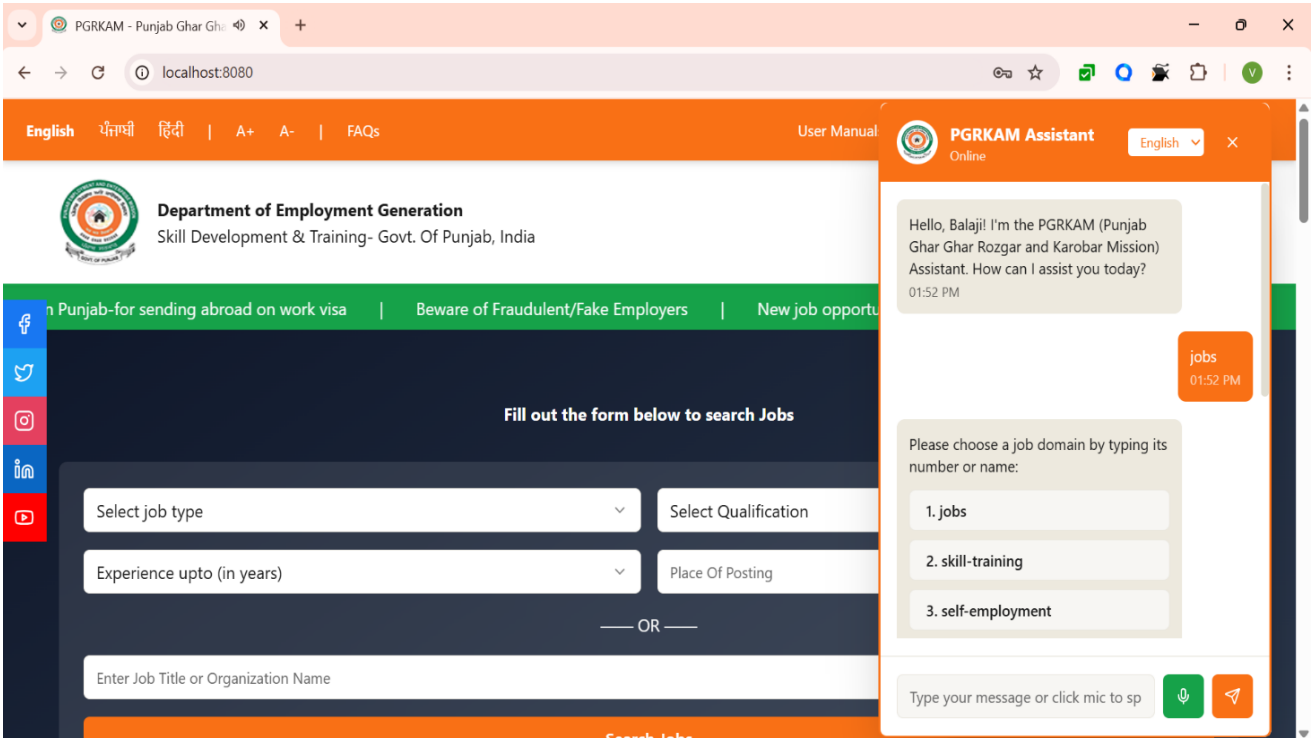
A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Conference Submission

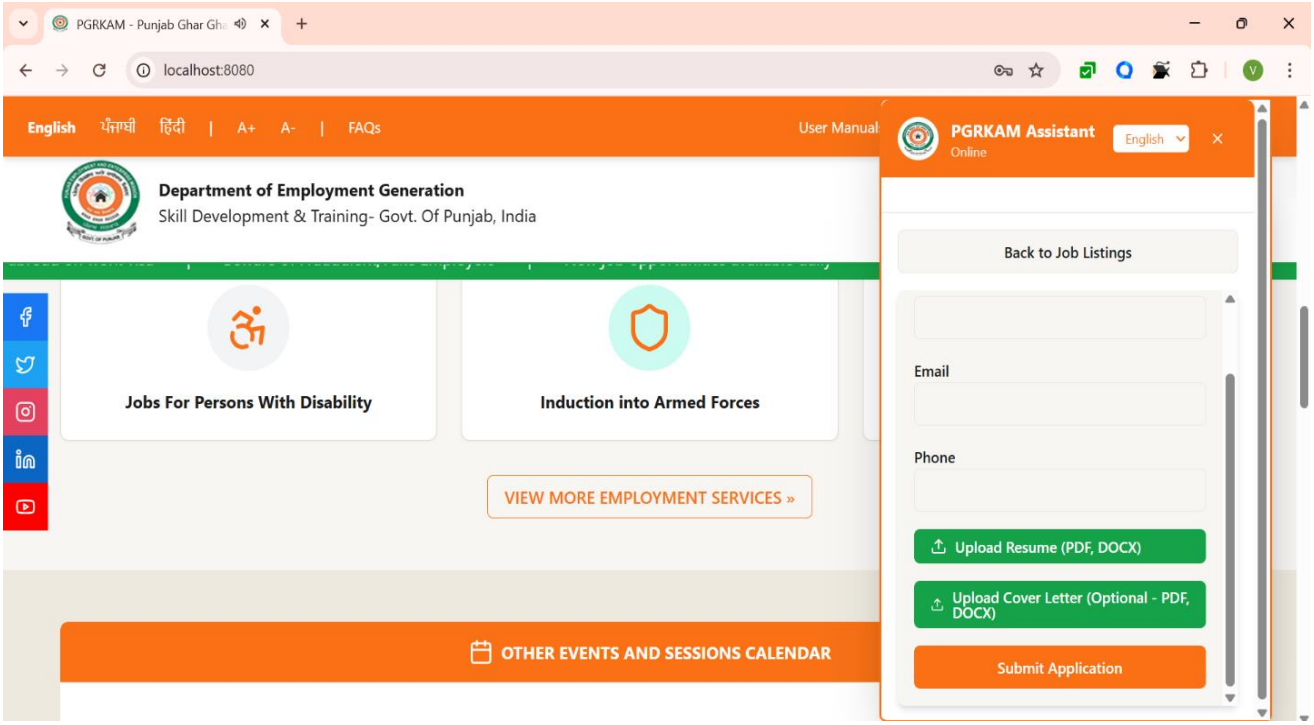


Project Output

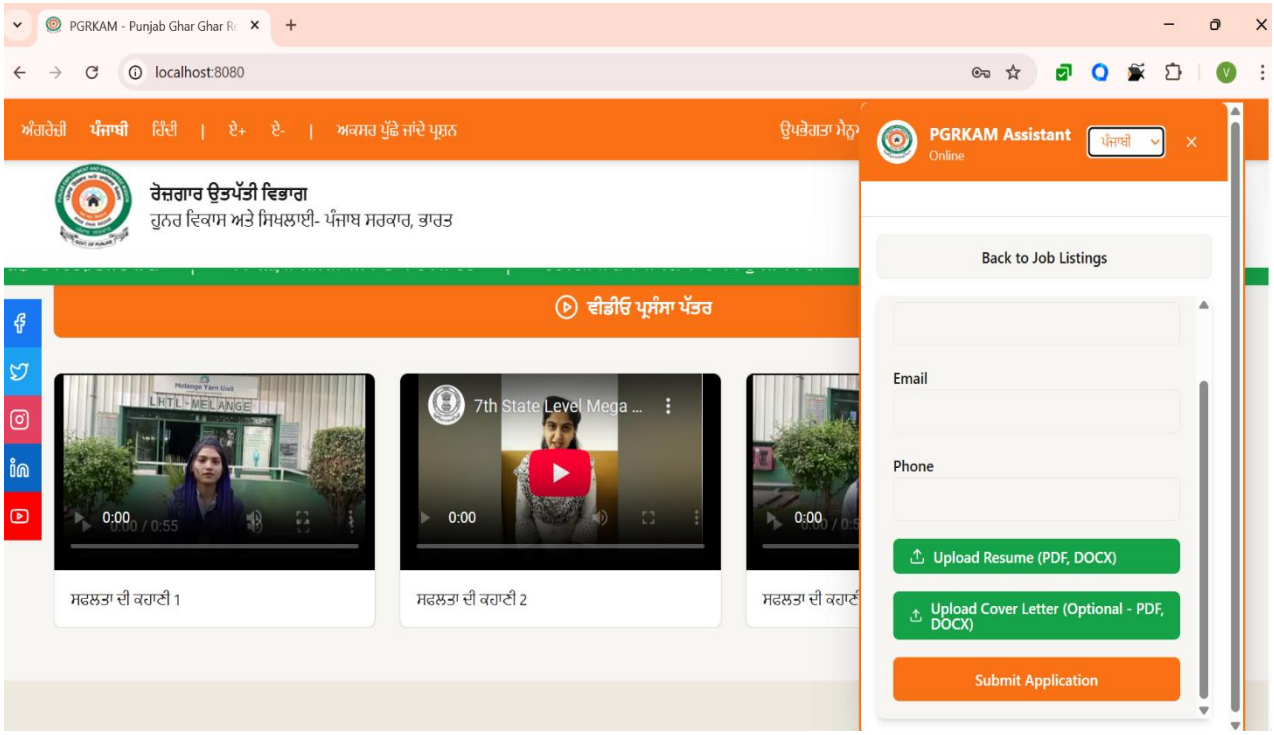
Home page/Login Page with Chatbot



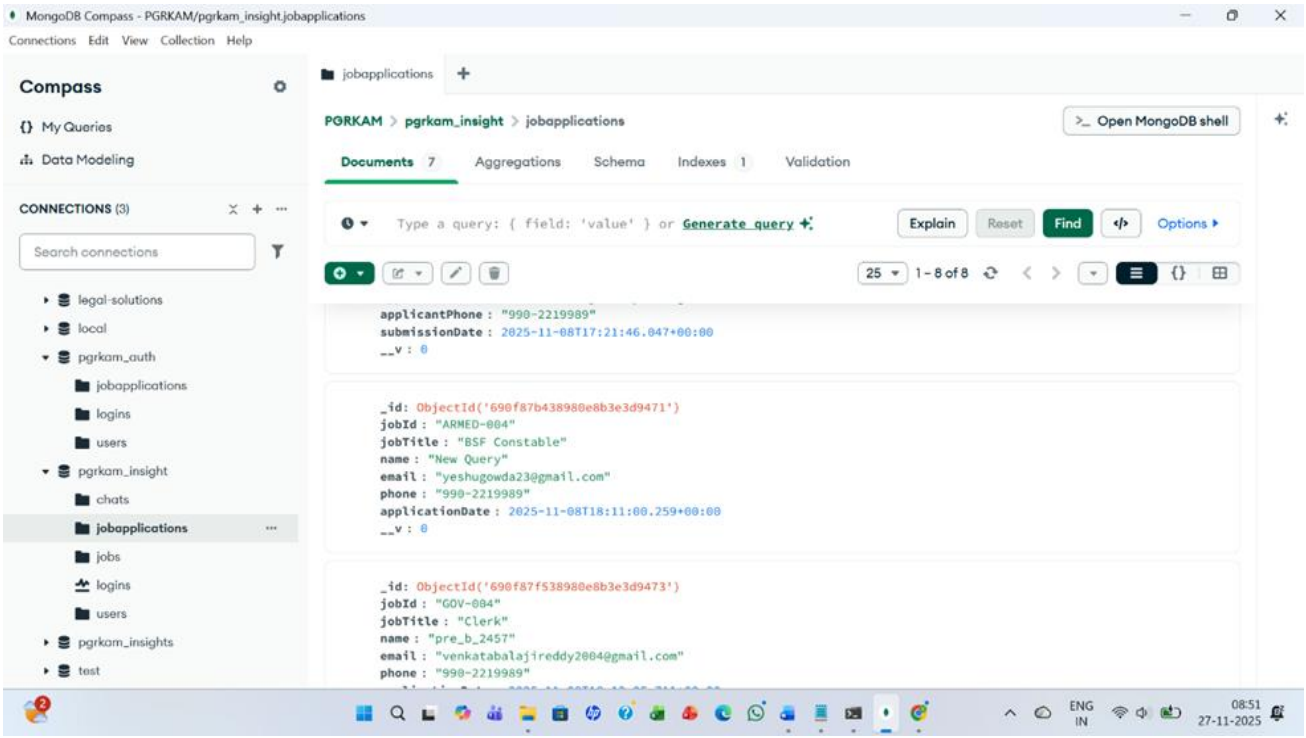
Jobs Search and Apply(a)



Jobs Search and Apply(b)



Backend (MongoDB)



Research paper Similarity(a)

turnitin Page 2 of 10 - Integrity Overview Submission ID trn:oid:::1:3416943756

1% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

- **6** Not Cited or Quoted 1%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

1% Internet sources
0% Publications
0% Submitted works (Student Papers)

Integrity Flags

1 Integrity Flag for Review

Replaced Characters
8 suspect characters on 1 page

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

Research paper Similarity(b)

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer
Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?
The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?
Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Github Link :https://vbalaji2004-pu.github.io/PGRKAM_CapstoneProject_CSE_62/

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

SMART NAVIGATION ASSISTANT FOR PGRKAM PORTAL

A PROJECT REPORT

Submitted by

V BALAJI- 20221CSE0727
SHREYA A- 20221CSE0720
NITHIN REDDY- 20221CSE0698

Under the guidance of,

Ms. RAMABAI ILAMURUGU
BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

PRESIDENCY UNIVERSITY

BENGALURU

NOVEMBER 2025

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report "SMART NAVIGATION ASSISTANT FOR PGRKAM PORTAL" is a Bonafide work of V BALAJI (20221CSE0727), SHREYA A (20221CSE0720), NITHIN REDDY (20221CSE0698) who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE ENGINEERING, during 2025-26.

Ms. Ramabai Ilamurugu	**Mr. Muthuraju V**	**Dr. Sampath A K**	**Dr. Blessed Prince**
Project Guide PSCS Presidency University	Program Project Coordinator PSCS Presidency University	School Project Coordinators PSCS Presidency University	Head of the Department PSCS Presidency University

Dr. Shakkeera L
Associate Dean
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS & PSIS
Presidency University

Examiners

Sl. no.	Name	Signature	Date
1.	Ms. Shaik Salma Begum		1/12/2025
2.	Mr. Suvetha K.H		1/12/25

PRESIDENCY UNIVERSITY
Private University Estd. in Karnataka State by Act No. 41 of 2013
Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named V BALAJI, SHREYA A and NITHIN REDDY hereby declare that the project work titled **"SMART NAVIGATION ASSISTANT FOR PGRKAM PORTAL"** has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

V BALAJI	20221CSE0727	V.Balaji
SHREYA A	20221CSE0720	Shreya
NITHIN REDDY	20221CSE0698	

PLACE: BENGALURU
DATE: 1/12/2025

Abstract

The PGRKAM, through its website and mobile application, offers a whole digital ecosystem with a wide spectrum of services including private and government jobs, self-employment opportunities, foreign employment, overseas education support, career counseling, induction into armed forces, and job melas. Despite this breadth, the platform lacks any sort of interactive guidance mechanism. Users, especially first-time visitors, rural citizens, and those with low digital or linguistic literacy, find it hard to reach the correct service module. Currently, the system expects such users to navigate manually through various interfaces, thereby confusing them, increasing query resolution time, and deteriorating the overall experience of access.

The proposed project envisages an advanced, multilingual AI-powered digital assistant to improve user interaction across the PGRKAM platform. The assistant is built in Java, React, and MongoDB, utilizing transformer-based LLMs for handling text and voice-based queries in Punjabi, Hindi, and English. It understands user intent, extracts relevant entities, and responds contextually related to job search, skill development, foreign counseling, and all other additional services offered on the platform. The system also has a built-in, multilingual speech recognition module and a screen reader that presents responses in audio format to support users with low literacy.

To make it more personalized, the interaction history, preference, and past searches of the user are kept in the Assistant's secure database to enable the system to recommend job opportunities and skill programs that best match the candidate's profile and interests for which the candidate is eligible. This assistant will also let users navigate easily to the module or service they are looking for, hence reducing the need for manual browsing and minimizing user effort.

The proposed solution ensures that conversational AI integrated with the existing infrastructure of the PGRKAM will allow citizens to have seamless conversations, listen, and explore various employment services on smartphones or computers with ease. This work improves the efficiency, usability, and inclusiveness of the PGRKAM digital platform and therefore forms a milestone in Punjab towards intelligent, citizen-centric digital governance.